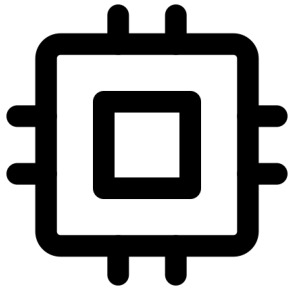
# MANGO BLOCK 入門

## 用 Python 控制你的開發板

Codpy Section



上午講者：蘇立原

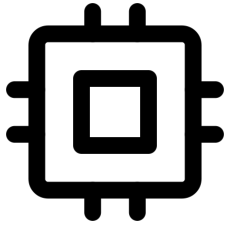
下午講者：鄭元凱

# 今日流程

| | | |
|---|---|---|
| 09:00 | 開場與設備介紹 | |
| 09:20 | 環境安裝 | 實作 |
| 10:00 | LED 與按鈕 | 實作 |
| 10:30 | 下課 | |
| 10:40 | pin 腳與蜂鳴器 | 實作 |
| 11:00 | 網頁儀表板實作 | 實作 |
| 11:45 | 展示與總結 | |
| 13:00 | 介紹、設定舵機 | 實作 |
| 14:30 | 舵機操控實作 | 實作 |

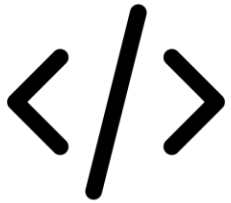
課程檔案

# 今天的目標



## 認識板子

彩燈、蜂鳴器、按鍵
在板子上的哪裡



## 用程式控制它

輸出：燈、聲音
輸入：按鈕



## 用網頁遙控它

藍牙網頁儀表板
中午前完成

# 先看成品


全部彩燈 on_led
選顏色 → 按「套用」，6 顆燈一起變色。
套用
全關
單顆彩燈 on_led
先選顏色，再點一顆圓點，只有那一顆燈會變色。
燈光效果 on_effect
呼吸、跑馬燈會使用目前設定的顏色。
靜態
彩虹
呼吸
跑馬燈
警示燈
亮度 on_brightness
16 %
蜂鳴器 tone_on / tone_off
按琴鍵，板子的蜂鳴器發出聲音。
Do
Re
Mi
Fa
Sol
La
Si
Do+
按鍵狀態 read_keys
按下板子上的 Key1 / Key2，網頁上對應的方塊要亮起來。
KEY1
KEY2
小遊戲：反應力 on_game_start / game_result
按「開始」後板子亮紅燈，等燈變綠就按 Key1，越快越好；還沒變綠就按算偷跑。（要先完成 TODO 7）
開始
反應時間：264 ms

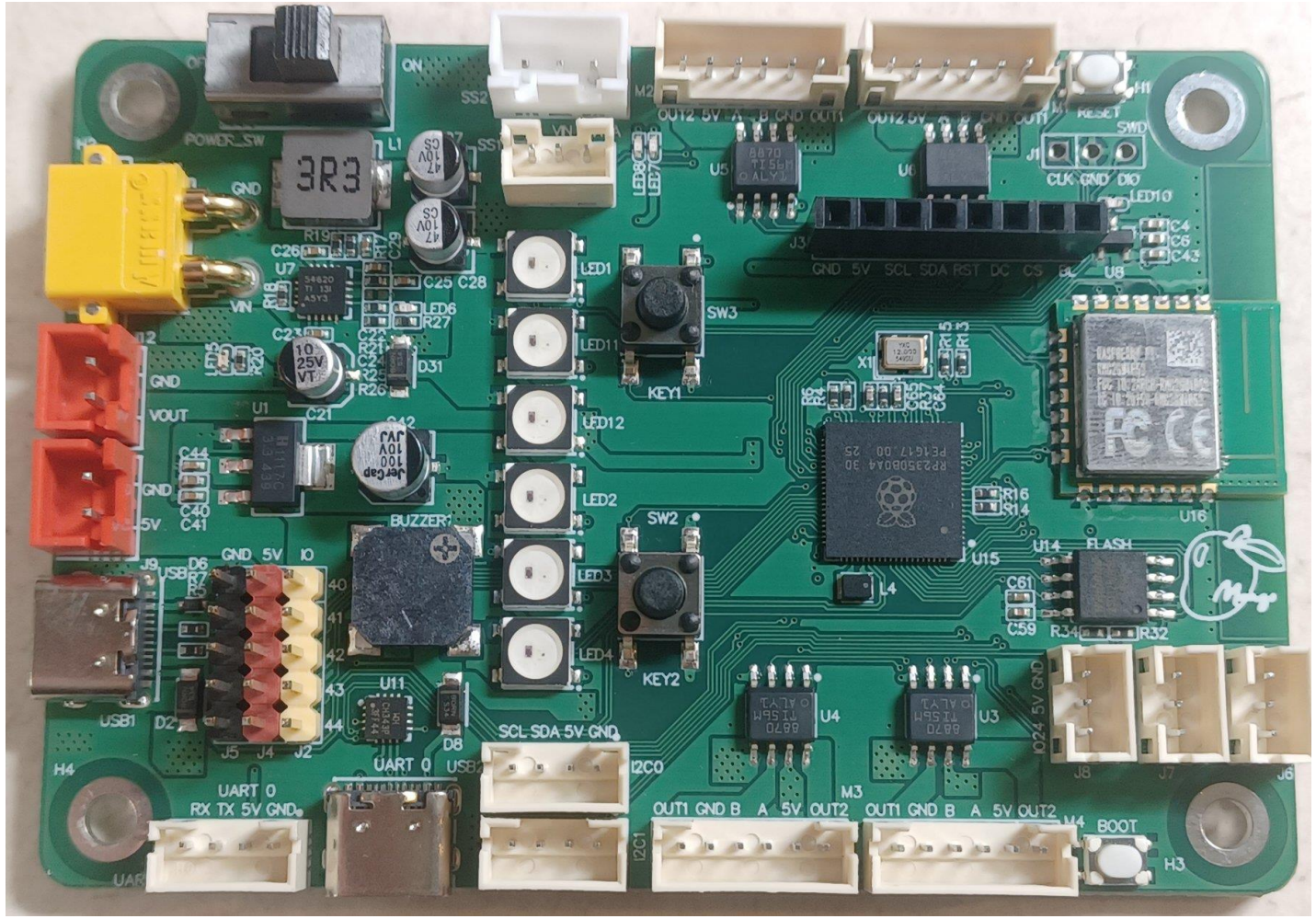
Part 0

# 認識 MANGO BLOCK

板子上有什麼

09:05 – 09:20

# 這就是 MANGO BLOCK




## MANGO BLOCK

動力驅動板

一塊板子 = 一台小電腦

可以跑 Python ( MicroPython )

可以用藍牙、Wi-Fi

可以控制舵機 ( 下午 )

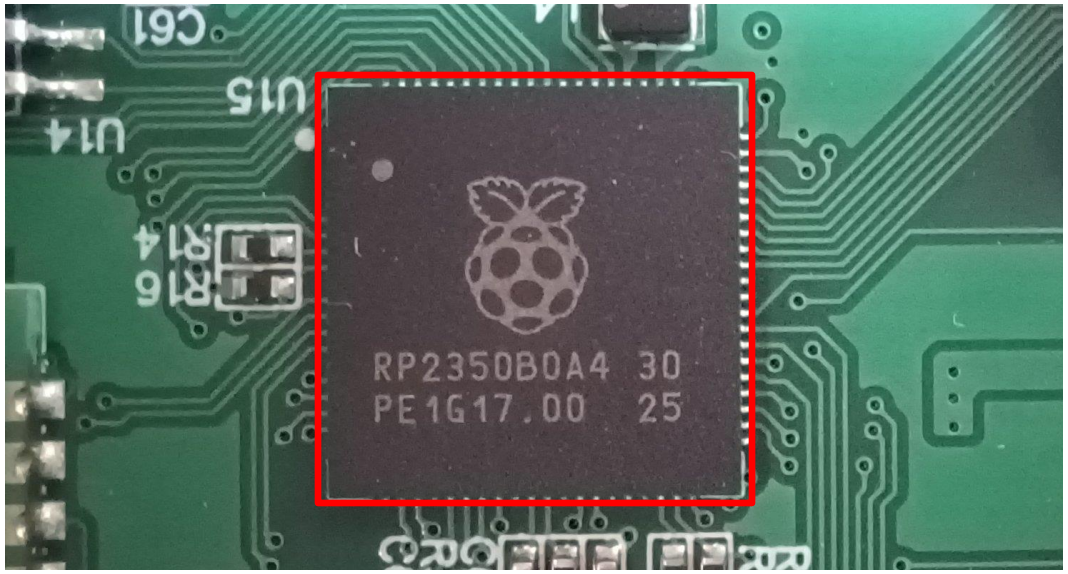
# 兩顆主要的晶片

## RP2350B

大腦

執行你寫的程式

控制燈、蜂鳴器、按鍵、舵機

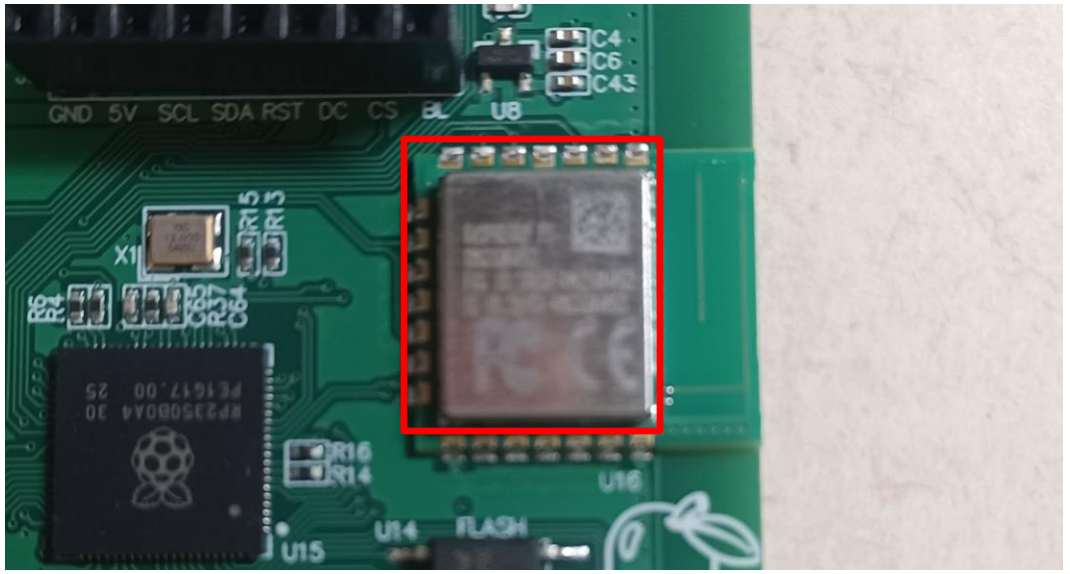



## RM2

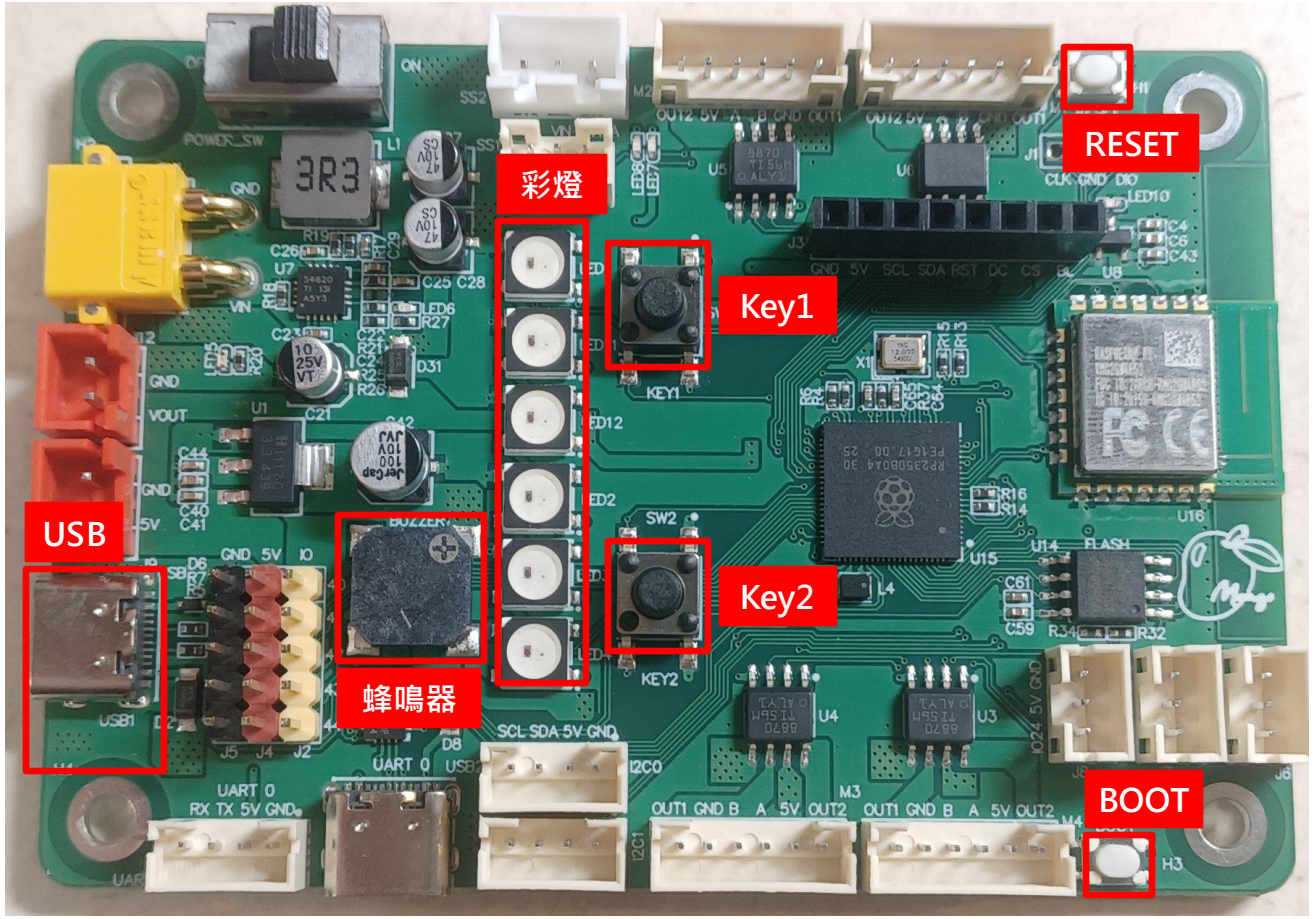
嘴巴和耳朵

藍牙、Wi-Fi 無線通訊

今天的專題靠它和網頁連線



# 今天會用到的元件




## 6 顆彩燈

WS2812B，可以變任何顏色

## 蜂鳴器

發出不同音高

## Key1 / Key2

兩顆給你用的按鍵

## BOOT / RESET

燒錄韌體、重新開機

## USB Type-C

供電 + 傳程式

Part 1

# 環境設定

下載 · 燒錄韌體 · VS Code · MicroPico

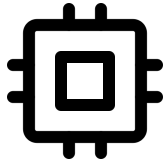
09:20 – 10:00

# 要準備的東西



課程檔案

git → Download ZIP



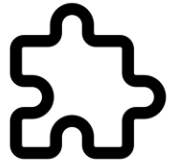
課程韌體

課程版 .uf2 檔



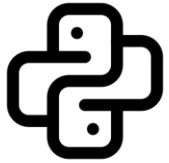
VS Code

寫程式的地方



MicroPico

VS Code 擴充功能
連到板子上的 Python



Python

儀表板實作開網頁用

# 下載課程檔案

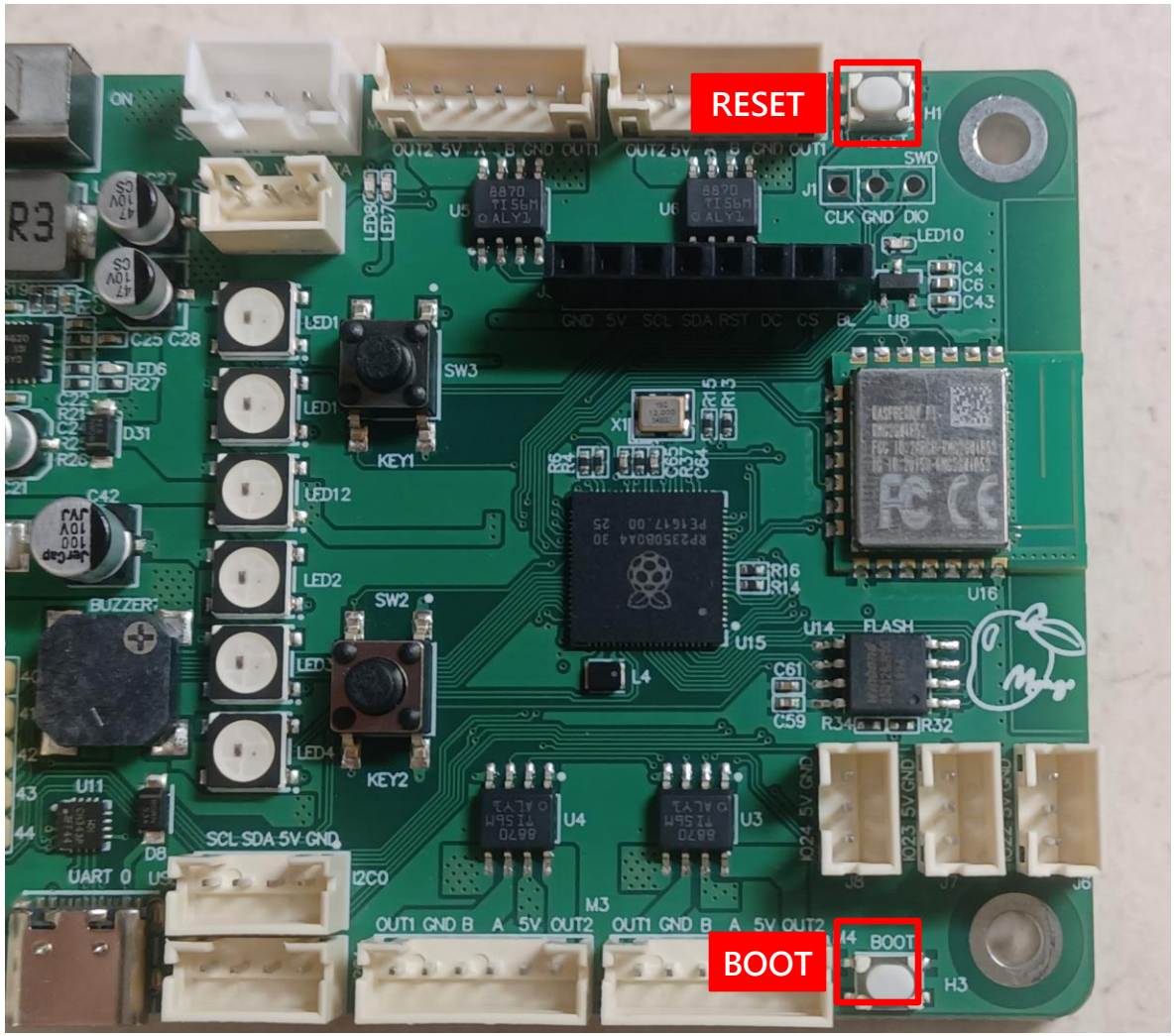



1

## Code → Download ZIP

解壓縮到桌面
資料夾裡有 morning、firmware

# 刷韌體




1. 按住 BOOT，按一下 RESET
   再放開 BOOT

   電腦會出現 RP2350 磁碟

2. 把韌體拖進 RP2350 磁碟

   ```
   firmware/MANGO_X4-v1.29.0.uf2
   ```

3. 磁碟自動消失 = 完成

   只要做一次，板子上的檔案不會被清掉

# 安裝 VS Code 與 MicroPico

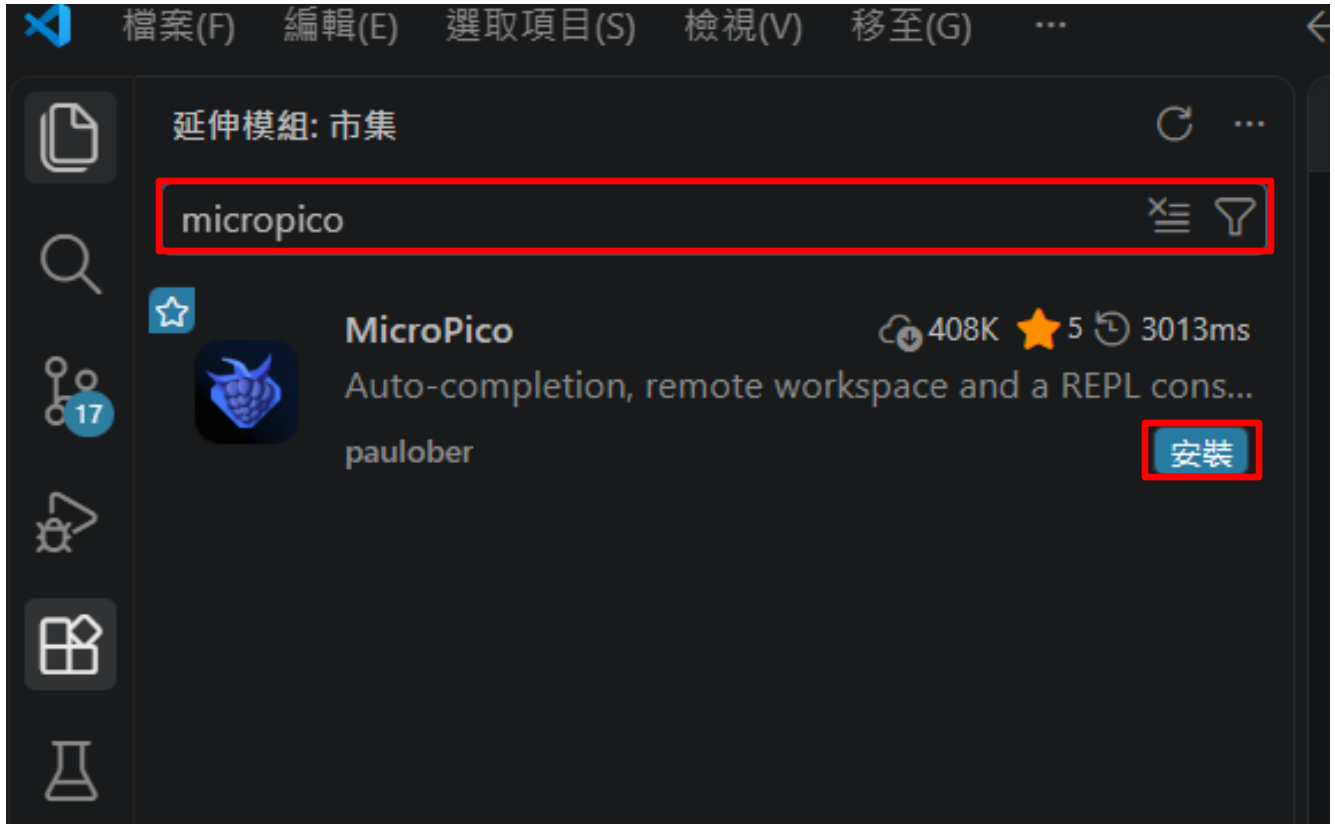



1

## 安裝 VS Code

code.visualstudio.com

# 安裝 VS Code 與 MicroPico




2

## 搜尋 MicroPico
## 按安裝

左邊「擴充功能」或 Ctrl+Shift+X
它建議的 Python 擴充功能也一起裝

# 安裝 VS Code 與 MicroPico



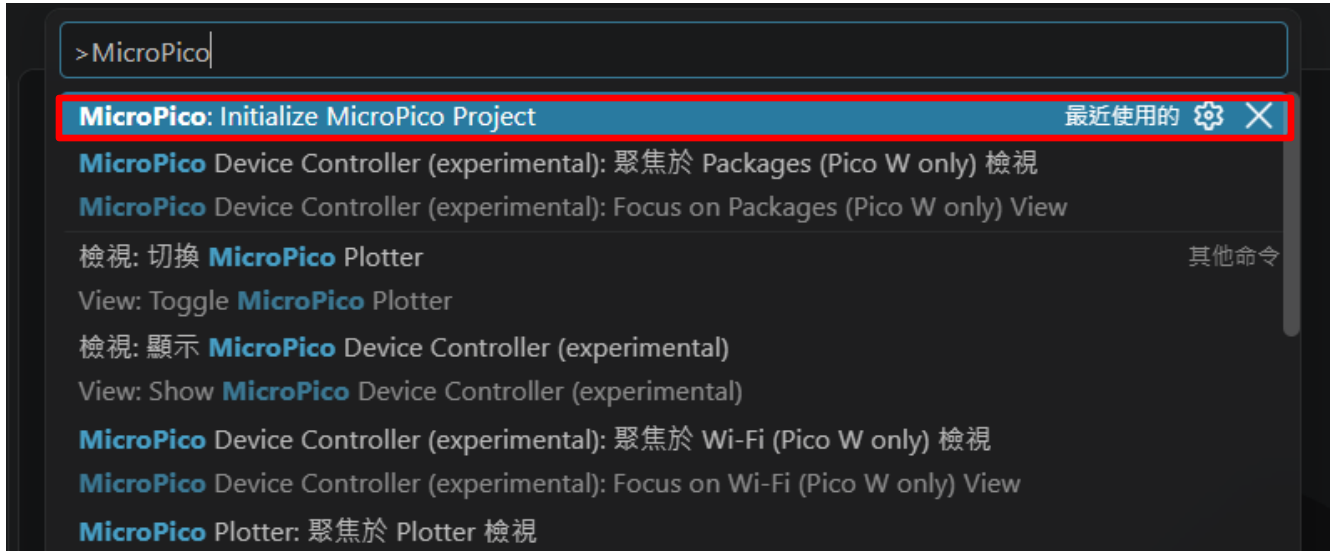

3

## 安裝 Python

python.org/downloads
下載 Windows 安裝檔

**一定要勾 Add python.exe to PATH**

# 安裝 VS Code 與 MicroPico



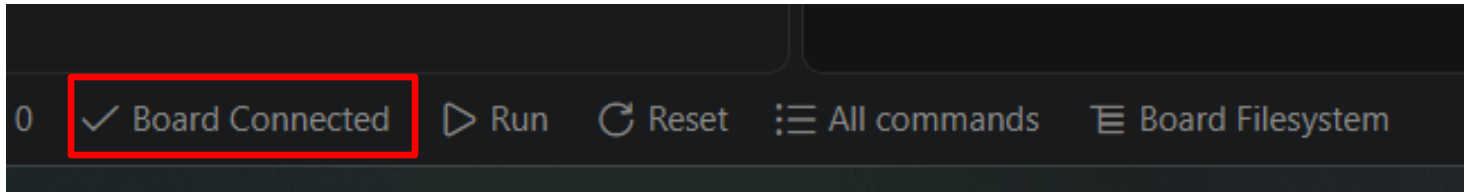

4

## 開資料夾
## 初始化專案

檔案 → 開啟資料夾 → 課程資料夾
Ctrl+Shift+P → 輸入 MicroPico

**MicroPico: Initialize MicroPico Project**

# 連上了嗎？




左下角顯示

## Board Connected

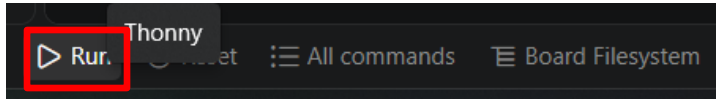
就是連上了

### 一直 Disconnected？

換一條能傳資料的線 · 關掉 Thonny 等佔用序列埠的程式 · 拔掉再插

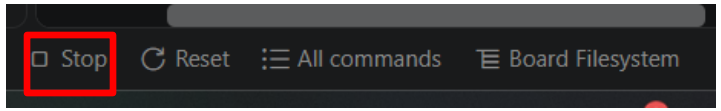
# 三個動作

## Run




把打開的檔案送到板子執行
不會存到板子

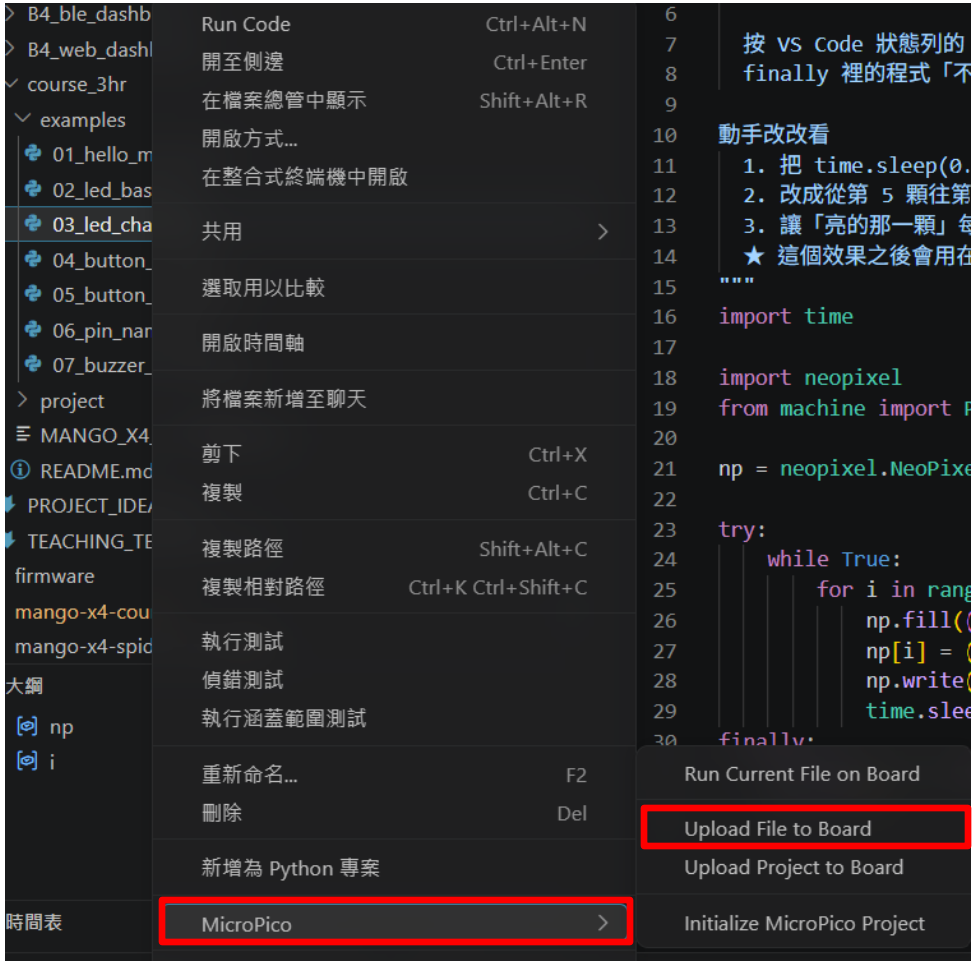
## Stop




停止有 while True 的程式
也可以在終端機按 Ctrl+C

## Upload

檔案按右鍵 → MicroPico → Upload File to Board
儀表板的 mango_ble.py 要這樣做



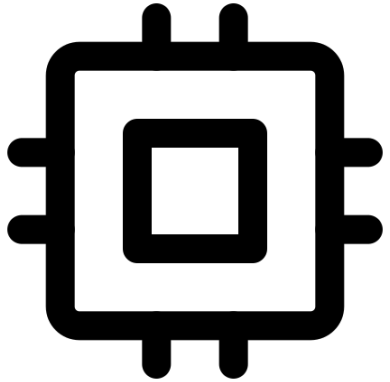
實作 01

# 板子點名

```
morning/examples/01_hello_mango.py
```

## 動手改改看

1. 打開檔案 → 按 Run
2. 看 6 顆燈亮起、嗶一聲
3. 已完成的同學：改改看燈的顏色



檢查點 終端機出現「環境 OK！」

實作 01

# 10 分鐘

燈亮了、嗶了、看到「環境 OK！」

Part 2

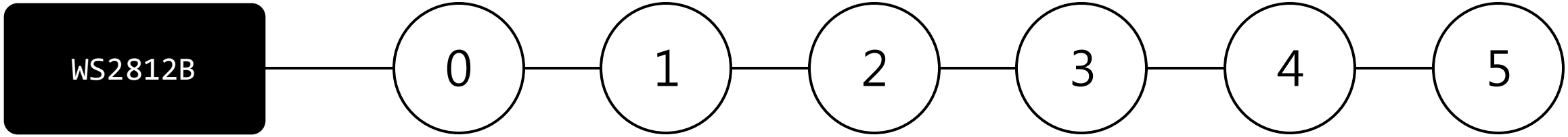
# LED 與按鈕

輸出・輸入

10:00 – 10:30

實作 02

# 6 顆燈，接在同一支腳上

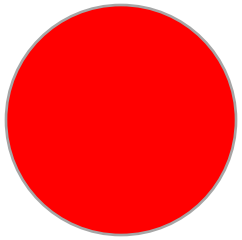



編號從 0 開始：0 ~ 5

```
np = neopixel.NeoPixel(Pin("WS2812B"), 6)
```

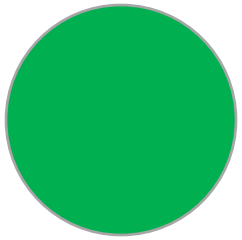
實作 02

# 顏色 = (紅, 綠, 藍)



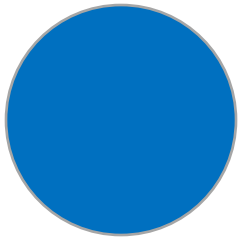
紅

(30, 0, 0)



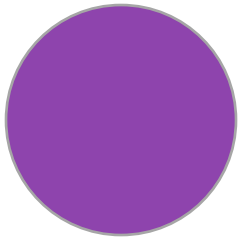
綠

(0, 30, 0)



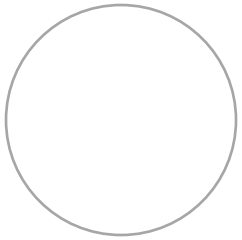
藍

(0, 0, 30)



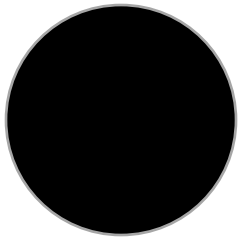
紫

(20, 0, 20)



白

(30, 30, 30)



關

(0, 0, 0)

## 0 ~ 255

每個數字的範圍

## 30

就夠亮了，255 太刺眼

實作 02

# 先記下來，write() 才送出

```
np[0] = (30, 0, 0)
```

→

```
np.write()
```

只是記在板子的記憶體裡
燈還不會變

這一行才真的把顏色
送到燈上

刪掉 write()，燈就不會變

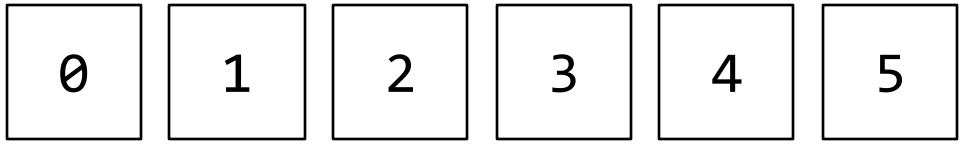
實作 02

# 用 for 迴圈控制 6 顆

02_led.py

```
for i in range(6):
    np[i] = (0, 0, 30)
np.write()
```

i 會依序是




6 顆都設定完，最後 write() 一次

全部同一色也可以寫

```
np.fill((0, 0, 30))
```

實作 02

# 控制彩燈

```
morning/examples/02_led.py
```

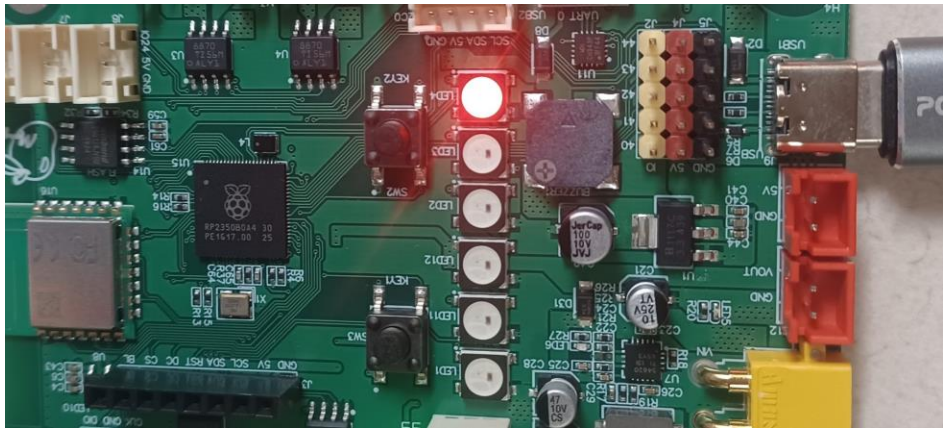
一顆 → 全部（for）→ 跑馬燈（一格一格畫，finally 關燈）

## 動手改改看

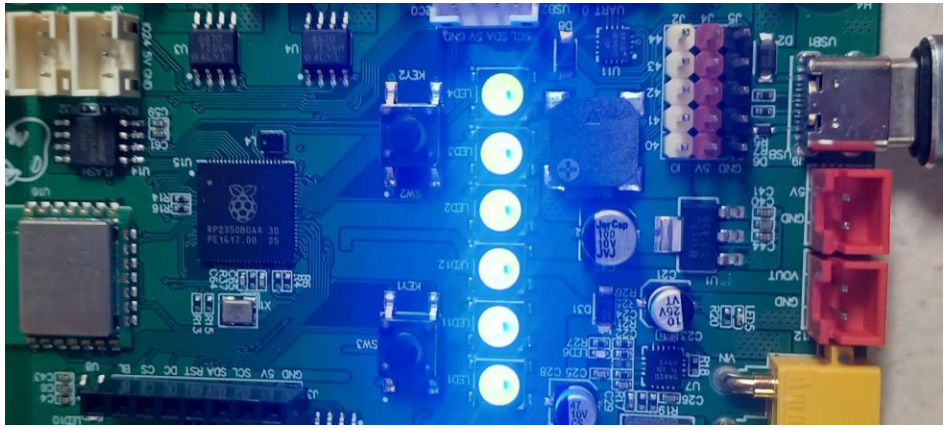
1. 第 0 顆改綠色；刪掉第一個 np.write() 會怎樣？
2. 偶數號紅、奇數號藍（i % 2 == 0）
3. 跑馬燈：sleep 改 0.3、0.03，或倒著跑

檢查點 能點亮一顆、設定全部，做出自己的跑馬燈

★ 會用在儀表板的 TODO 1，跑馬燈會用在 TODO 4~6



一顆：第 0 顆亮紅



全部：6 顆亮藍

實作 02

# 10 分鐘

實作 03

# 讀按鈕：一行設定，一行讀取

設定

```
key1 = Pin("KEY1", Pin.IN, Pin.PULL_DOWN)
```

↑ 哪一支腳
按鍵 1

↑ 輸入
讀進來

↑ 沒按時固定是 0
下一頁解釋

讀取

```
key1.value()
```

→ 1 按下　0 放開

實作 03

# 為什麼要 PULL_DOWN？

## 沒加

沒按的時候，腳位沒接到任何東西
讀到的數字會亂飄

```
0 1 1 0 1 0 0 1
```

## 加 Pin.PULL_DOWN

沒按的時候，把腳位「拉」到 0
數值才會穩定

```
0 0 0 0 1 1 0 0
```

板子上的按鍵一定要加

實作 03

# 一直讀，一直判斷

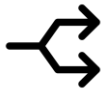
03_button.py

```
while True:
    pressed = key1.value()
    if pressed == 1:
        np.fill((30, 0, 0))
    else:
        np.fill((0, 0, 0))
    np.write()
    time.sleep(0.1)
```



讀按鈕

value() → 0 或 1



判斷

1 亮紅燈，0 關燈



送出

np.write()



等一下，重複

每 0.1 秒讀一次

程式跑得很快，看起來就像「按住才亮」

實作 03

# 那「按一下開、再按一下關」呢？

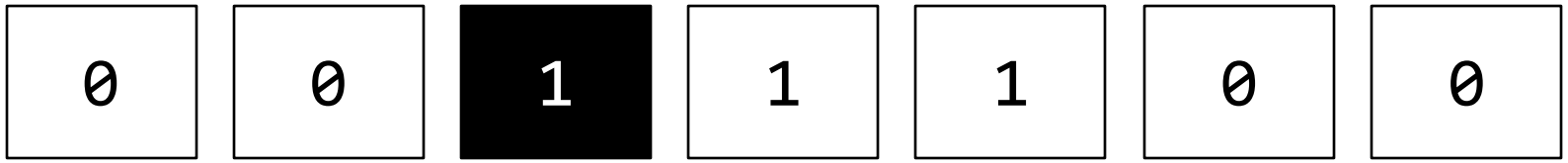
## Key1 按住才亮

看「現在」是 0 還是 1

## Key2 按一下切換

看「從 0 變成 1」的那一瞬間

讀到的值




↑ 上一次 0、這一次 1 → 剛剛被按下！

03_button.py

```
if last == 0 and now == 1:
    light_on = not light_on
```

`time.sleep(0.02)`

去彈跳：等按鈕金屬片停止彈跳
按一次才不會被算成好幾次

實作 03

# 按住才亮、按一下切換

```
morning/examples/03_button.py
```

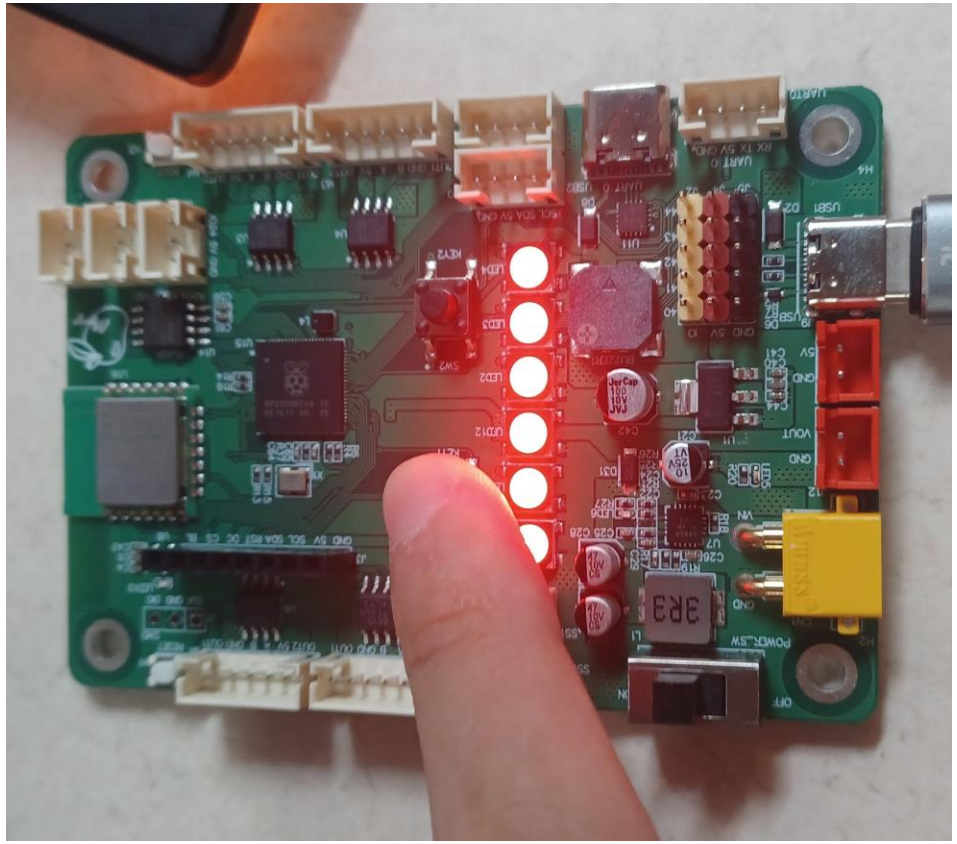
Key1：看「現在」　Key2：抓「從 0 變成 1」

## 動手改改看

1. 按住 Key1 改成亮藍燈
2. sleep(0.02) 改成 0，快速按 Key2
3. 每按一次 Key2 換色：紅 → 綠 → 藍

檢查點　Key1 按住才亮、Key2 能切換開關燈

★ 讀按鈕用在儀表板的 TODO 2，「剛剛被按下」用在 TODO 7



實作 03

# 10 分鐘

# 休息一下

10:30 – 10:40

Part 3

# pin 腳與蜂鳴器

名字 ↔ 號碼 · 音高 · 音量

10:40 – 11:00

# 每個元件都接在一支腳上

| 名字 | GPIO | 接到 |
|---|---|---|
| WS2812B | 45 | 6 顆彩燈 |
| BUZZER | 46 | 蜂鳴器 |
| KEY1 | 30 | 按鍵 1 |
| KEY2 | 31 | 按鍵 2 |

GPIO = 晶片上可以自己控制的腳

# 名字和號碼是同一支腳

```
Pin("KEY1")
```

好讀、不容易寫錯

=

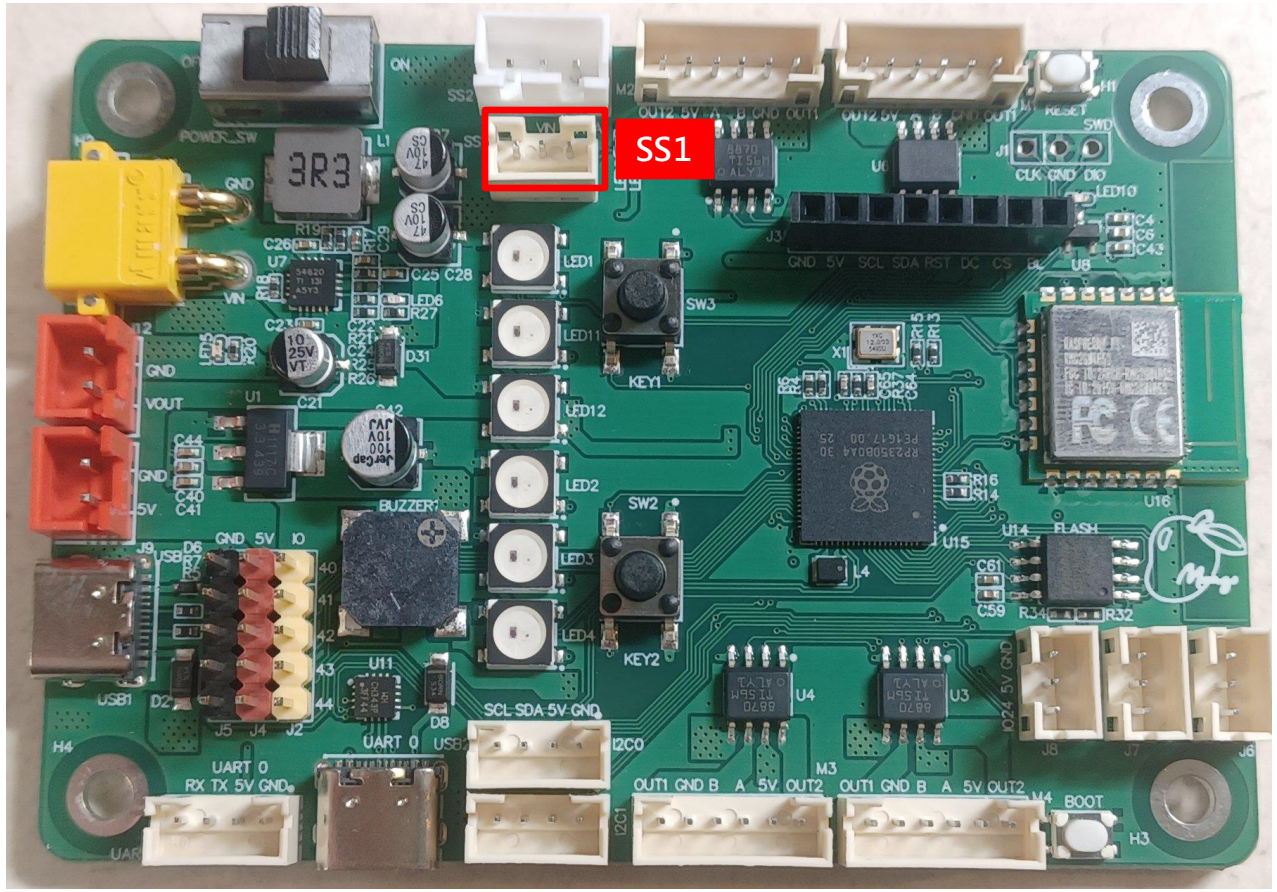
```
Pin(30)
```

要背號碼

課程韌體幫每支腳取了名字，今天程式裡一律用名字

板子上所有腳位：morning/PINS.md（參考用，不用實作）

# 下午會用到的接頭




## SS1

接 Hiwonder 匯流排舵機

| 名字 | GPIO | 用途 |
|---|---|---|
| SS_TX | 8 | 送封包給舵機 |
| SS_RX | 9 | 讀回舵機的回覆 |

一條線串很多顆，用封包 + ID 指定誰
UART · 115200 baud

實作 04

# 其實 01 已經嗶過了

01_hello_mango.py

```
buzzer = PWM(Pin("BUZZER"))

buzzer.freq(880)          # 音高
buzzer.duty_u16(3000)     # 音量（開始發聲）
time.sleep(0.2)           # 響 0.2 秒
buzzer.duty_u16(0)        # 音量 0 = 停止
```

1. 發聲
   `freq + duty_u16`
2. 等一下
   `sleep`
3. 停止
   `duty_u16(0)`

蜂鳴器的用法就這 4 行，打開 01 找找看

沒有 duty_u16(0) → 程式停了還一直響

實作 04

# 蜂鳴器只要記兩個數字

## freq

頻率 = 音高

數字越大聲音越高
440 = La

## duty_u16

音量

3000 左右就夠大聲
0 = 安靜

實作 04

# 音高對照

| Do | Re | Mi | Fa | Sol | La | Si | Do⁺ |
|---|---|---|---|---|---|---|---|
| 262 | 294 | 330 | 349 | 392 | 440 | 494 | 523 |

網頁儀表板上的琴鍵，送來的就是這些數字

實作 04

# 歌譜是資料，程式只負責播

資料：歌譜

```
song = [
    (DO, 0.4), (DO, 0.4),
    (SOL, 0.4), (SOL, 0.4),
    (LA, 0.4), (LA, 0.4), (SOL, 0.8),
]
```

(音高, 長度)

音高 0 = 休止符

程式：播放器

```
for pitch, length in song:
    發聲 pitch
    響 80% 的時間
    安靜 20%
```

為什麼要安靜 20%？

兩個一樣的音才分得開
Do Do 不會黏成一個長 Do

換一首歌 = 只改資料，不用改程式

實作 04

# 小星星

```
morning/examples/04_buzzer_melody.py
```

## 動手改改看

1. 補完後半段：Fa Fa Mi Mi Re Re Do
2. 換成你自己會的一首歌
3. 每播一個音，就亮一顆彩燈



檢查點 能播放自己改過的旋律

實作 04

# 10 分鐘

Part 4

# 網頁儀表板實作

網頁已完成，你寫板子端

11:00 – 11:55

# 今天要做的系統



網頁儀表板 → 藍牙 → mango_ble.py → dashboard.py → 燈 · 蜂鳴器 · 按鍵

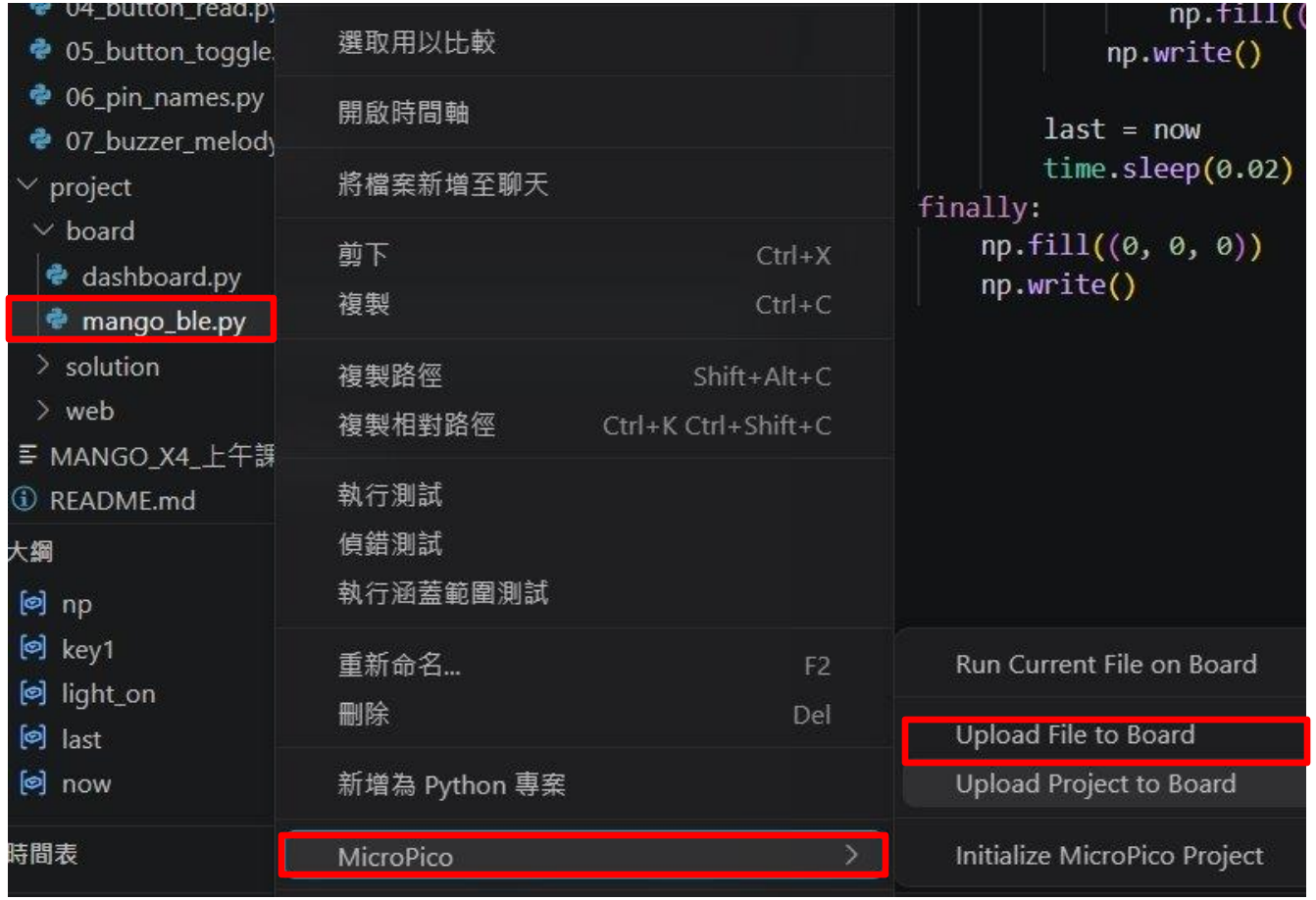
**已完成** | **你寫的** | **01 ~ 04 學過的**

藍牙的原理今天不講，交給工具庫 mango_ble.py

# 網頁按一下 → 你的函式被呼叫

整個實作只要記住這一句

# 準備步驟




1

## 上傳工具庫

mango_ble.py 按右鍵
→ Upload File to Board
只要做一次

# 準備步驟




2

## Run dashboard.py
## 記下自己的名字

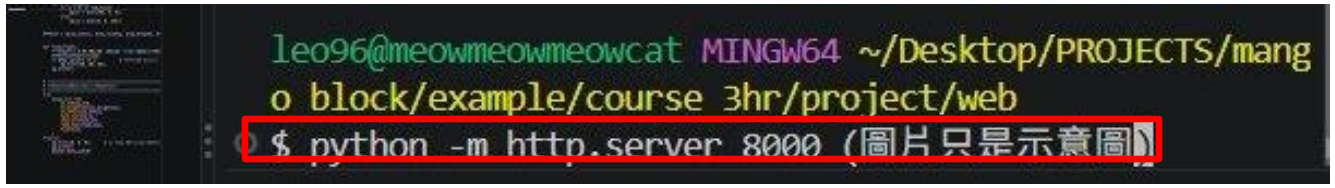
教室裡有 30 塊板子
要連到自己的那塊

```
MANGO-XXXX
```

# 準備步驟




① 終端機右上角 + → 新增終端機




② 輸入指令

3

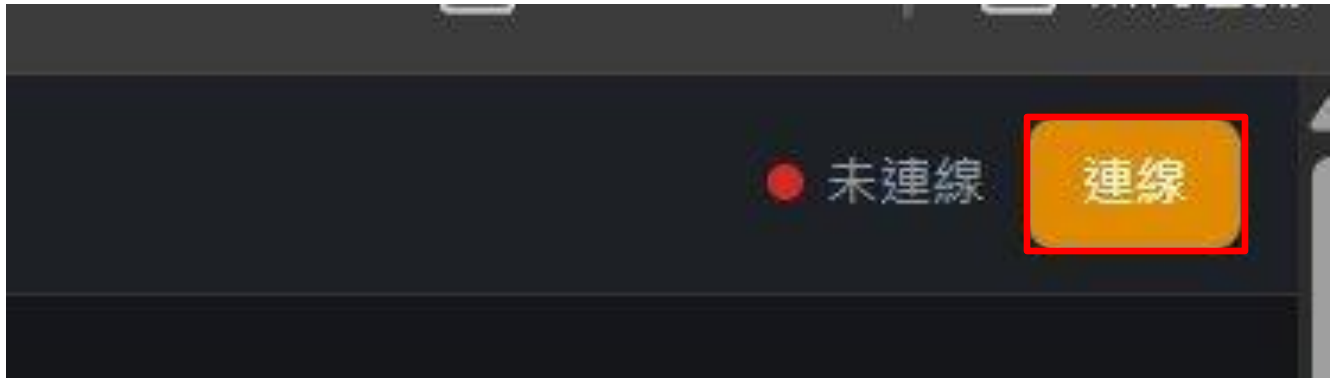
## 開網頁伺服器

VS Code → Terminal → New Terminal

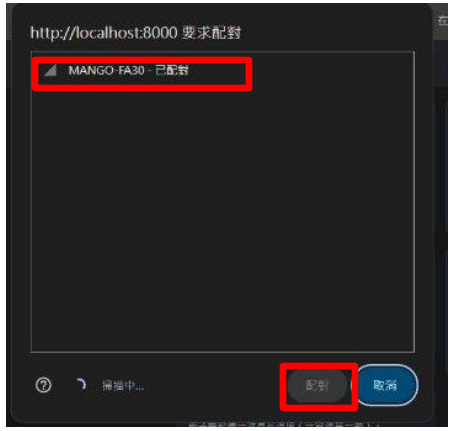
```
cd morning/project/web
python -m http.server 8000
```

找不到 python ? 改用 py -m http.server 8000

# 準備步驟




① 按右上角「連線」




② 選自己的 MANGO-XXXX → 配對

4

## 連到自己的板子

Chrome / Edge 開
localhost:8000
→ 連線 → 選 MANGO-XXXX

**檢查點：網頁顯示「已連線」**

改完程式：Stop → Run → 網頁重新連線

# 先看懂範例 A

dashboard.py

```
def on_brightness(value):
    global brightness
    brightness = value

def on_effect(number):
    global effect
    effect = number
```

拖動亮度滑桿、按效果按鈕，終端機會看到

```
[網頁] on_brightness(55)
[網頁] on_effect(3)
```

所有 TODO 都是這個模式：網頁呼叫你的函式 → 你存起來或直接控制

網頁傳值進來 → 存進變數

其他地方（tick）會用這些變數畫燈

**global**

要修改外面的變數
先宣告 global

# TODO 1：on_led

必做　和 02 的 for 迴圈一樣

網頁選了顏色
index 0 ~ 5 只改那一顆、255 改全部

```
def on_led(index, r, g, b):
    # 255 → 6 顆都改
    # 其他 → colors[index] = (r, g, b)
    ...
```

測試 「全部彩燈」選色按套用、點「單顆彩燈」圓點

## 和 02 比一比

```
for i in range(6):
    np[i] = (0, 0, 30)
np.write()
```

把 np[i] 換成 colors[i]
顏色換成網頁送來的 (r, g, b)

不用 write()：tick 會把 colors 畫到燈上

# TODO 2：read_keys

必做　和 03 讀按鈕一樣

回傳兩顆按鍵的狀態
要回傳兩個值

```
def read_keys():
    return 0, 0   # ← 改成讀真正的按鍵
```

測試 按住板子 Key1 / Key2，網頁方塊亮起

## 和 03 比一比

```
key1 = Pin("KEY1", Pin.IN,
           Pin.PULL_DOWN)
pressed = key1.value()
```

key1、key2 已經幫你設定好了

回傳兩個值的寫法：
return 第一個, 第二個

# TODO 3：tone_on / tone_off

必做　和 01 的「嗶一聲」一樣

按網頁琴鍵 → 蜂鳴器發聲
響多久由工具庫計時

```
def tone_on(freq):
    # 用 freq 開始發聲

def tone_off():
    # 停止發聲
```

測試 按網頁琴鍵；連線/斷線時也會嗶一聲

## 和 01 比一比

```
buzzer.freq(880)
buzzer.duty_u16(3000)
time.sleep(0.2)
buzzer.duty_u16(0)
```

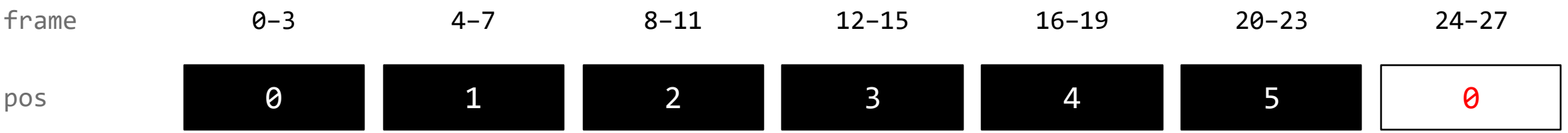
880 換成 freq
前兩行放 tone_on、最後一行放 tone_off

sleep 不用寫：工具庫會計時

# TODO 4：draw_chase

必做　和 02 的跑馬燈幾乎一樣

frame 每 0.03 秒 +1，你只要畫「第 frame 格」




```
pos = (frame // 4) % 6
# 第 pos 顆用 dim(colors[pos])，其他 (0, 0, 0)
```

燈是黑的？先用「全部彩燈」設一個顏色

### 和 02 比一比

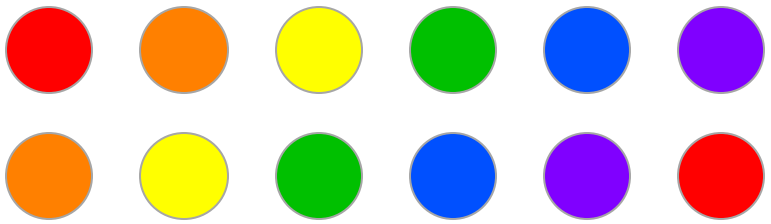
```
np.fill((0, 0, 0))
np[i] = (0, 30, 0)
```

i 換成 pos；sleep、write 交給工具庫

# TODO 5：draw_rainbow

**挑戰** 02 的跑馬燈 + 串列

6 顆燈各用 RAINBOW 的一個顏色，整排一直往前流動




← 每 4 格整排往前移一個

```
shift = frame // 4
for i in range(6):
    # 第 i 顆用 RAINBOW[(i + shift) % 6]，記得 dim()
```

測試 網頁按「彩虹」

## 進階：wheel(pos)

```
c = wheel(i * 40 + frame * 4)
np[i] = dim(c)
```

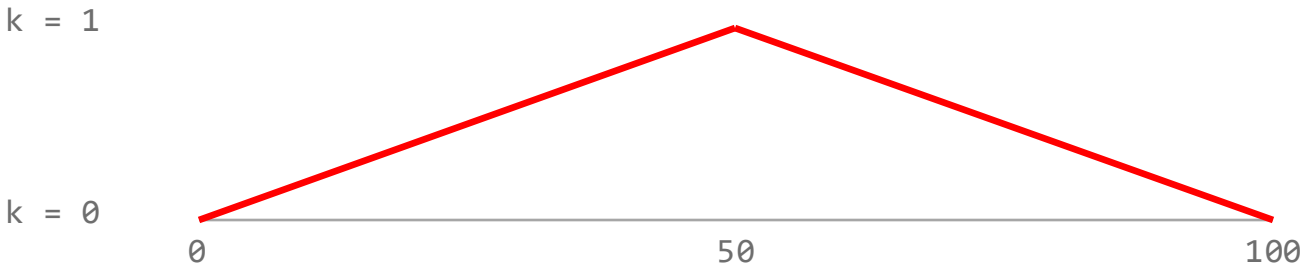
dashboard.py 裡已經寫好 wheel()：把 0~255 變成彩虹上的顏色，可以做出平滑漸層。

不想用 wheel？直接在 RAINBOW 定義 6 個顏色就好，顏色也可以自己換。

# TODO 6：draw_breathe

**挑戰** 02 的 for 迴圈 + 小數

一個循環 100 格：前 50 格慢慢變亮、後 50 格慢慢變暗




```
step = frame % 100
# 變亮 k = step / 50；變暗 k = (100 - step) / 50
# 每顆：c = dim(colors[i])，三個數字都乘上 k
```

測試 網頁先設一個顏色，再按「呼吸」

## 和 02 比一比

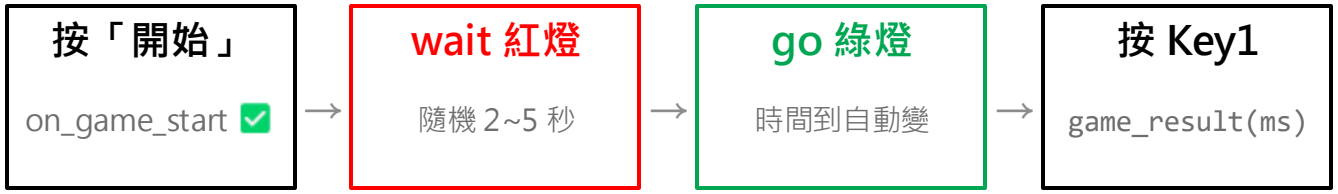
```
for i in range(6):
    np[i] = (0, 0, 30)
```

一樣用 for 迴圈設定 6 顆。

乘上 k 之後會變成小數，顏色一定要是整數：
(int(c[0] * k), int(c[1] * k), int(c[2] * k))
不然會出現 TypeError

# TODO 7：小遊戲：反應力

**挑戰** 和 03 的 Key2 一樣，抓「剛剛被按下」




綠燈前就按 = 偷跑 → game_result(-1)

```
def draw_game(frame):
    # ② wait 6 顆紅、go 6 顆綠：np.fill(dim(...))
    # ③ Key1 剛剛被按下？
    #    wait → mango_ble.game_result(-1)
    #    go   → mango_ble.game_result(
    #               time.ticks_diff(now, go_time))
```

測試 網頁「小遊戲：反應力」按開始

## 和 03 比一比

```
now = key1.value()
if last == 0 and now == 1:
    ...  # 剛剛被按下
last = now
```

最後記得更新 last_key1，按完 game_state 改回 "idle"。

反應時間由板子自己量，不會被藍牙延遲影響。

加分：mango_ble.beep(1320, 120)

# 網頁和程式的結構

| 檔案 | 負責什麼 | 什麼時候改 |
| --- | --- | --- |
| `board/dashboard.py` | 你的函式：收到指令後怎麼控制板子 | TODO 都在這裡 |
| `board/mango_ble.py` | 板子端藍牙：收網頁的資料、呼叫你的函式 | 新增藍牙功能（看檔案最上面的步驟） |
| `web/index.html` | 網頁上有哪些卡片、按鈕 | 加按鈕、加卡片 |
| `web/app.js` | 按下去做什麼：mango.write(...) | 按鈕的動作 |
| `web/mango-ble.js` | 網頁端藍牙：write / read / subscribe | 新增藍牙功能時加一行 UUID |
| `web/style.css` | 顏色、排版 | 改外觀 |

每個檔案開頭都有說明，自由發揮前先讀一讀

# 自由發揮：和 AI 一起做



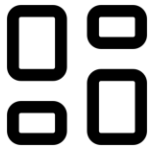
警示燈

效果 4 還空著
前 3 紅、後 3 藍互換



手電筒

按住 Key2
6 顆燈全白



改網頁

換顏色、改排版
加一張卡片

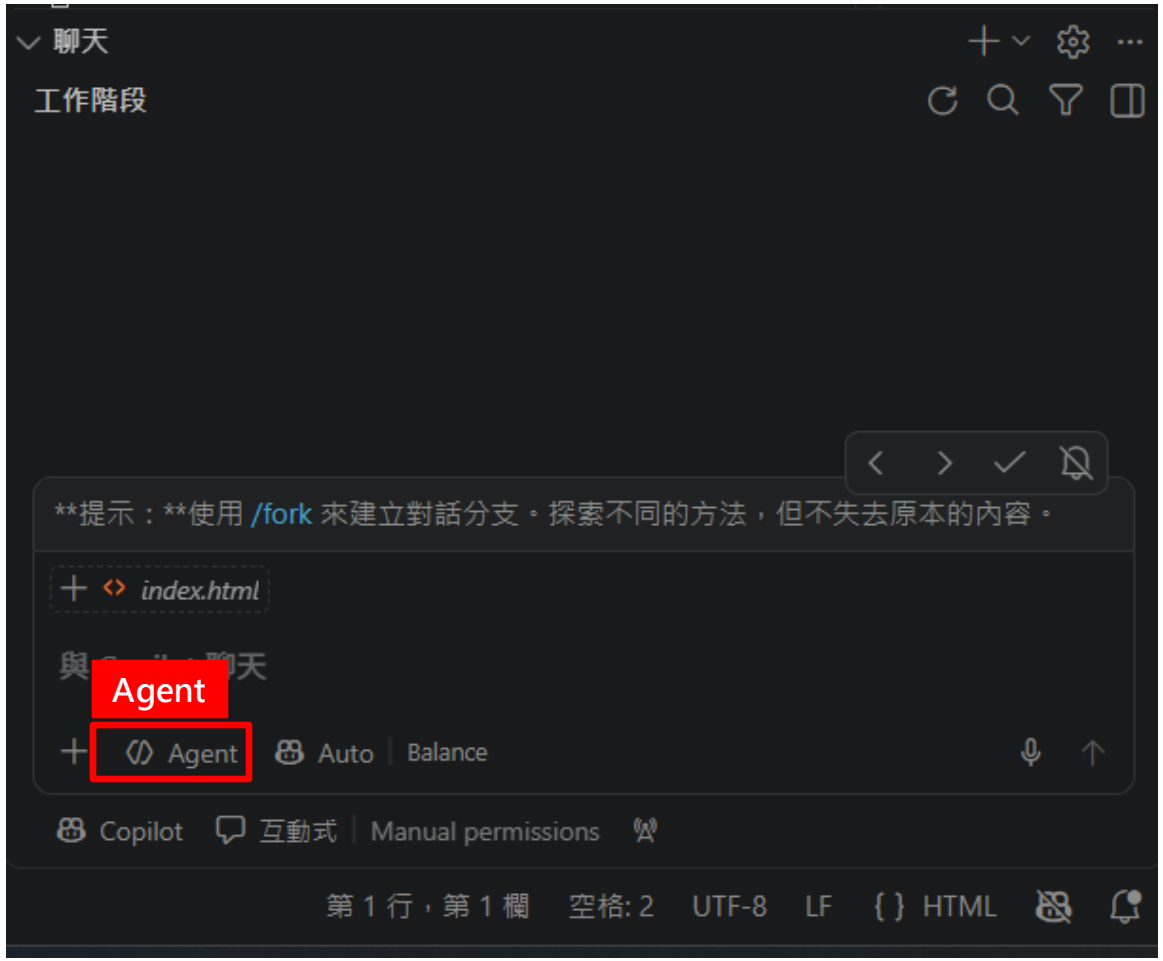


新功能

網頁多一個按鈕
板子多一個函式



# VS Code 內建的 AI Agent




1. **打開聊天**

   Ctrl+Alt+I，或右上角的聊天圖示

2. **模式選 Agent**

   Agent 會自己讀檔案、直接改程式

3. **描述你想要的效果**

   例如：網頁多一個按鈕，按下去 6 顆燈變白色

改完記得 Stop → Run → 網頁重新連線

網頁儀表板實作

# 30 分鐘

TODO 1~4 必做，做完挑戰 TODO 5~7，或和 AI 一起自由發揮

11:45

# 展示時間

2 ~ 3 位同學分享小遊戲或自由發揮的作品

# 今天學到的



## 輸出

彩燈：np[i]、write()
蜂鳴器：freq、duty_u16



## 輸入

按鈕：value()
PULL_DOWN、去彈跳



## 腳位名稱

Pin("KEY1") = Pin(30)



## 網頁遙控

網頁按一下
→ 你的函式被呼叫

午休

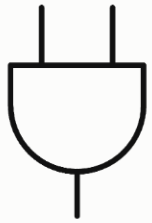
# 下午：舵機實作

MANGO BLOCK × Hiwonder 匯流排舵機

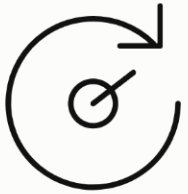
# 舵機實作

MANGO BLOCK × Hiwonder 匯流排舵機

# 目標



接好一顆



連上它
讓它轉



三顆一起

# 準備材料




Bus Servo x 3

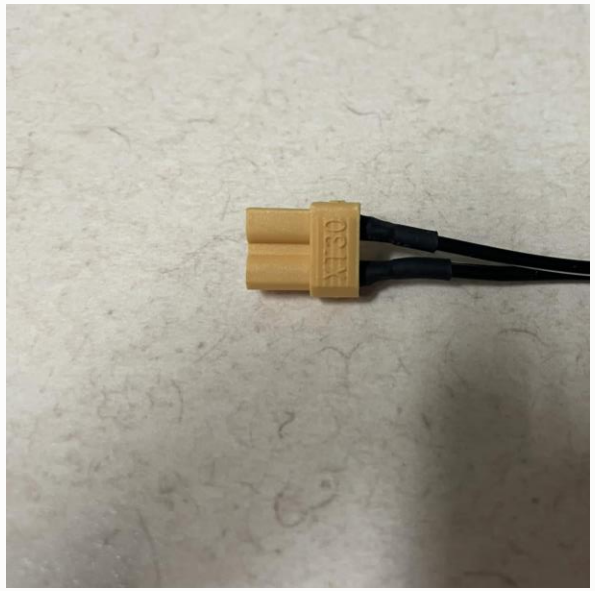


舵機串接線 x 3

# 準備材料

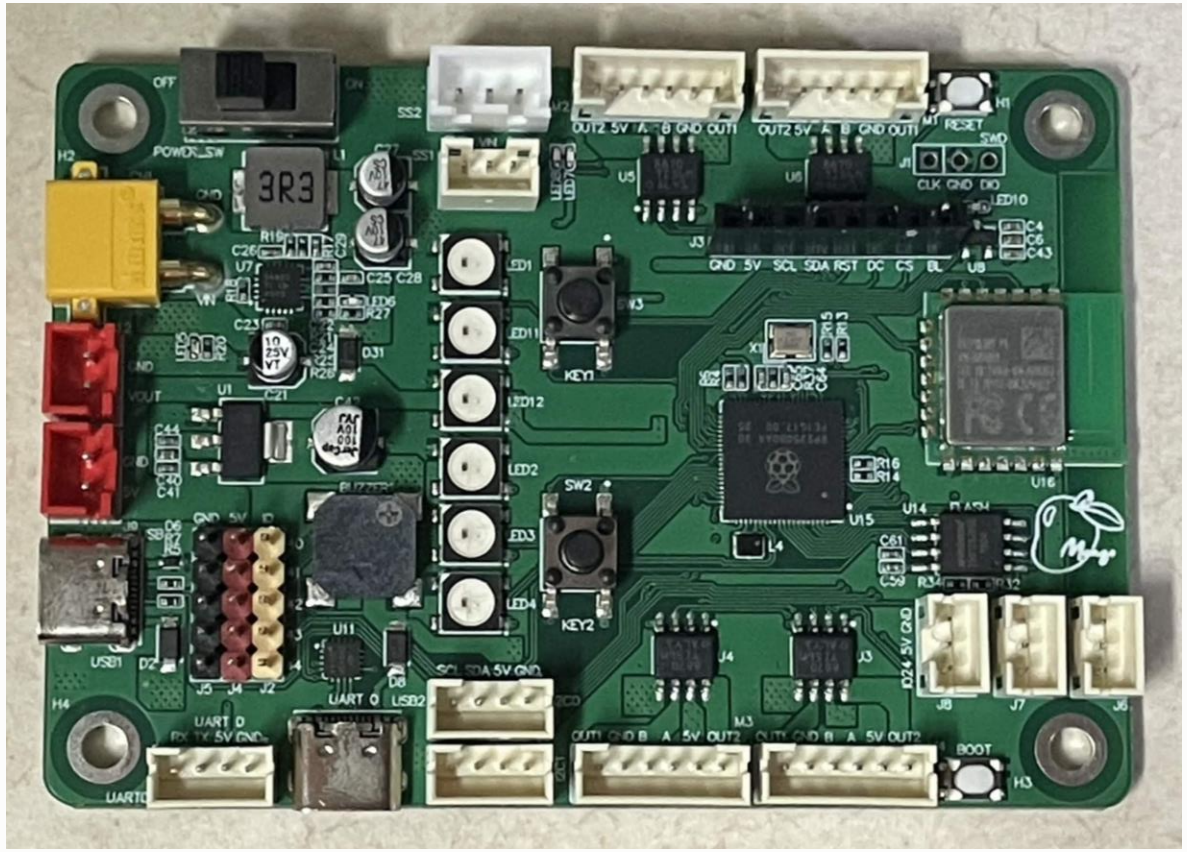


Type-C USB




XT30 供電線

# 準備材料




MANGO BLOCK x1

# 檢查清單

- [ ] MANGO BLOCK 板 × 1
- [ ] Bus Servo × 3
- [ ] 舵機串接線 × 3
- [ ] XT30 線
- [ ] Type-C 線

# 記得斷電

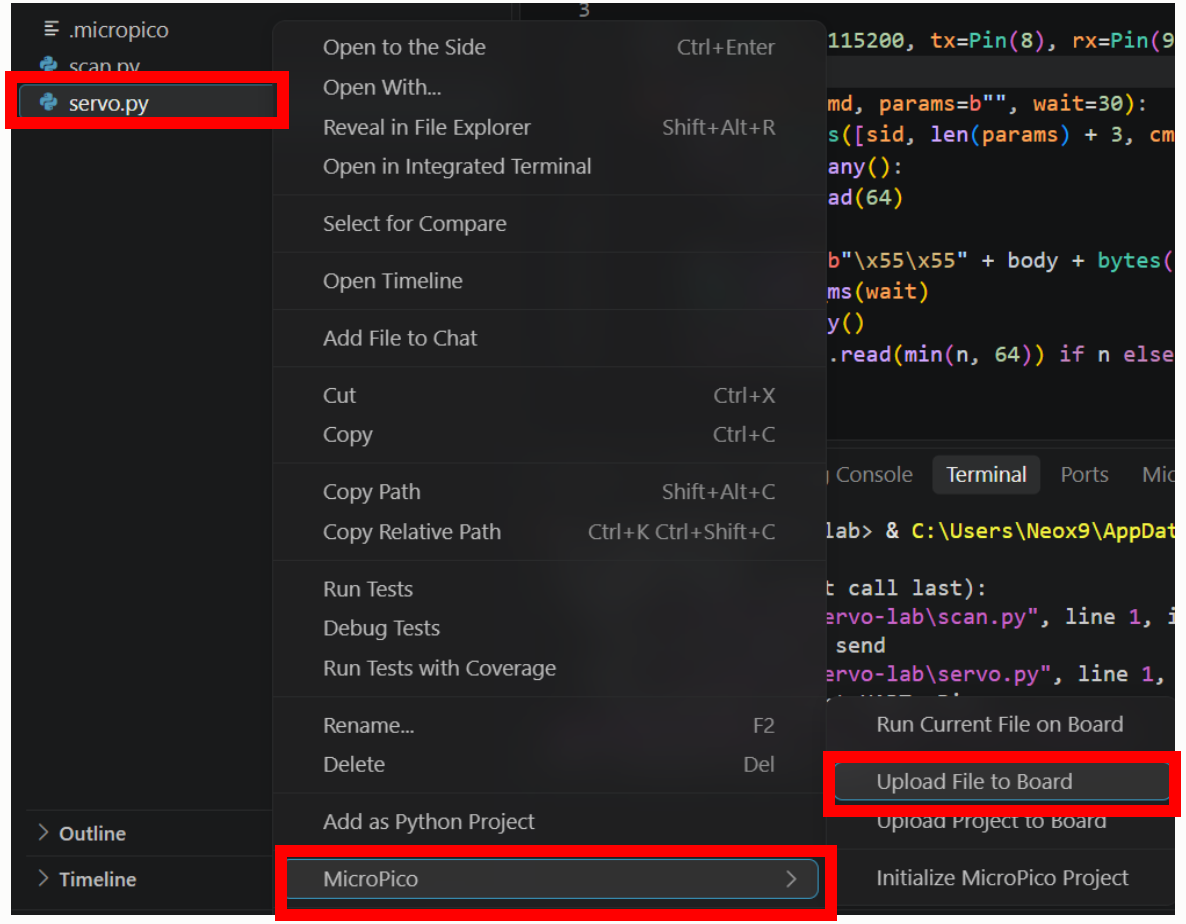
做任何電路上得動作時請把電源關掉或拔掉電源線

- [ ] 改接線之前，先關外接電源
- [ ] 舵機固定好、手離開再做動作
- [ ] 小角度、慢速的轉
- [ ] 注意角度限制
- [ ] 讀到亂碼或雜訊，不要送移動指令



# 透過 Vscode 跑程式






1

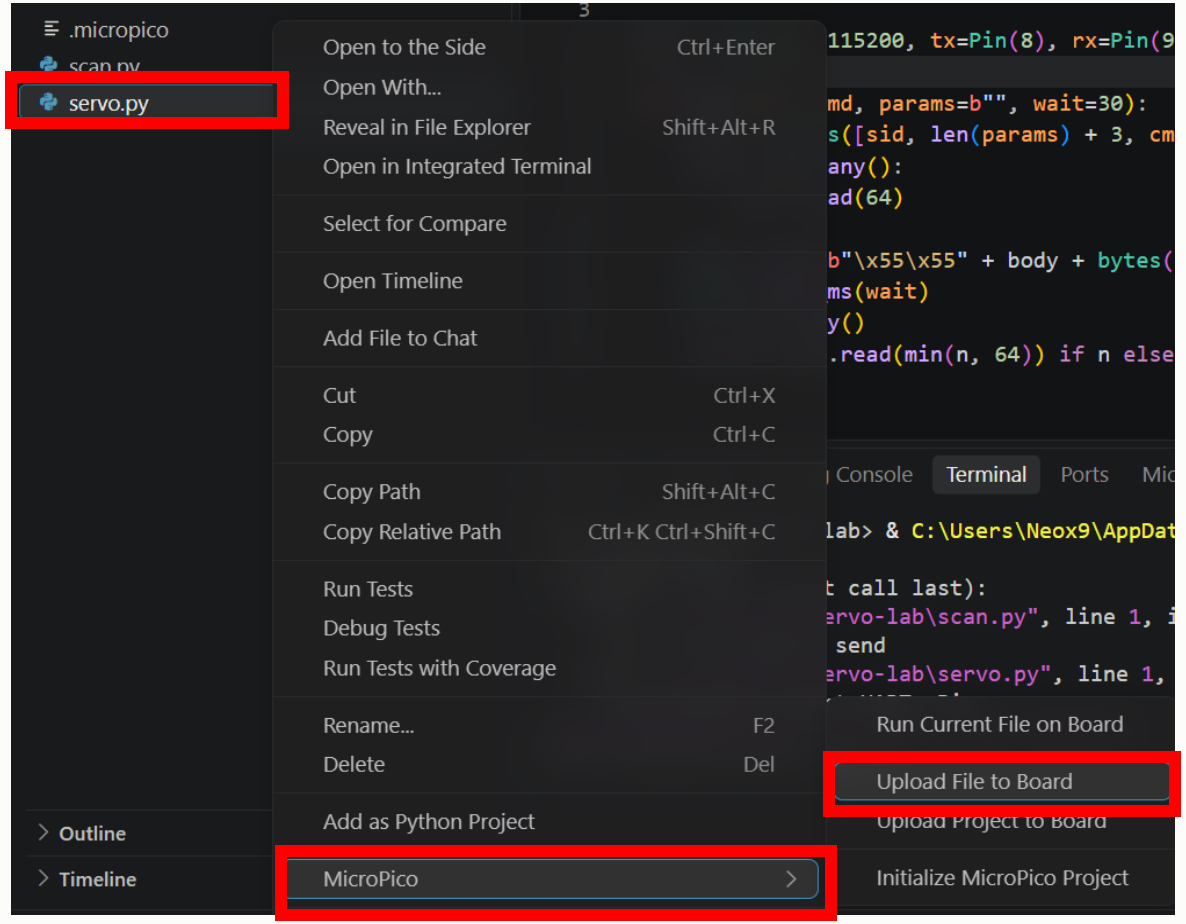
## 上傳 servo.py

在檔案上按右鍵

→ Upload file to Pico

# 透過 Vscode 跑程式






2

## 讓蜂鳴器叫

在 vREPL 輸入：

```
from servo import beep
beep(1)
```

Part 1

# 單顆舵機

接線 · 讀取 · 讓它轉

# 認識舵機






Hiwonder

## HTD-45H

45 kg·cm

高壓匯流排舵機

金屬齒輪 · 一條線控制、
也讀得回狀態

# 一般舵機 vs 匯流排舵機



## 一般 PWM 舵機

一顆佔一支腳

只能下命令

讀不到現在的角度

## 匯流排舵機

一條線串很多顆

用封包 + ID 指定誰

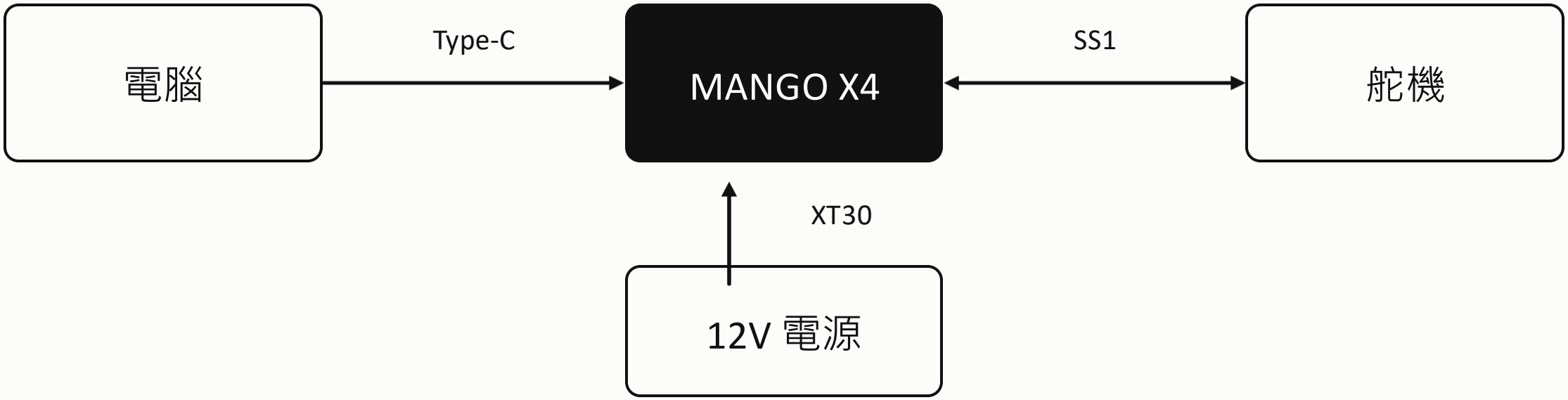
讀得到位置、電壓、溫度

# 規格



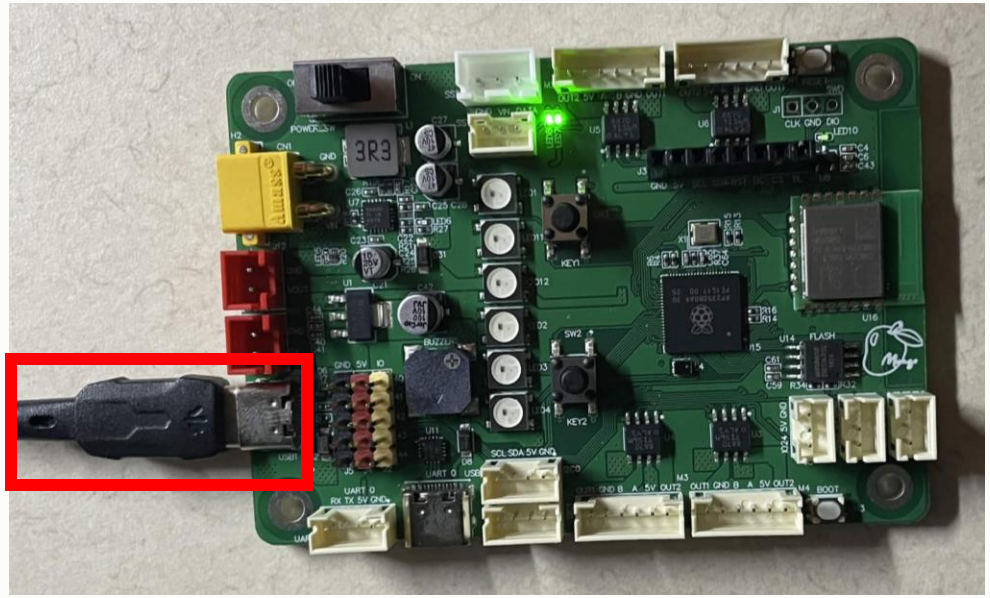
| | | |
|---|---|---|
| **45 kg·cm**<br>扭力 · 11.1V 時 | **9–12.6 V**<br>工作電壓 | **0–240°**<br>位置 0–1000 |
| **0.18 s**<br>轉 60° · 11.1V 時 | **ID 0–253**<br>出廠是 1 | **115200**<br>baud · 單線半雙工 |

# 接線總覽



# 接線





1

## 插上 Type-C

板子開機，VS Code 連上

# 接線



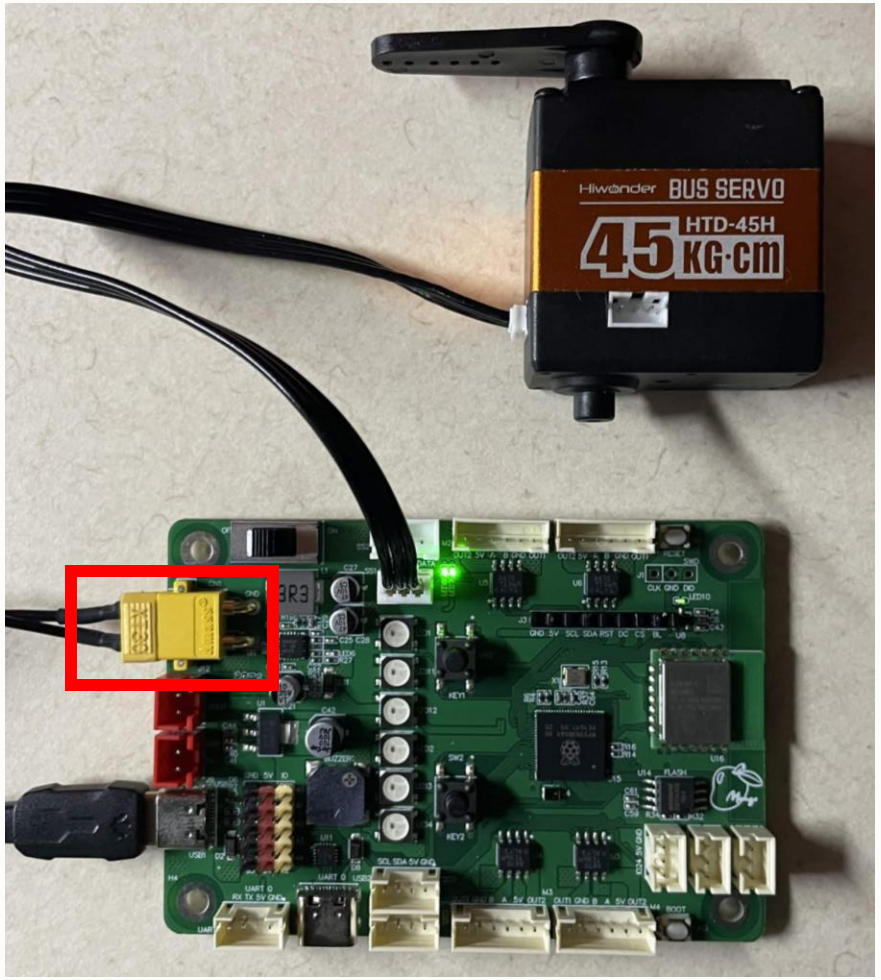



2

舵機連接 SS1

# 接線






3

## XT30 接 12V 電源

**開關先保持 OFF**

# 接線



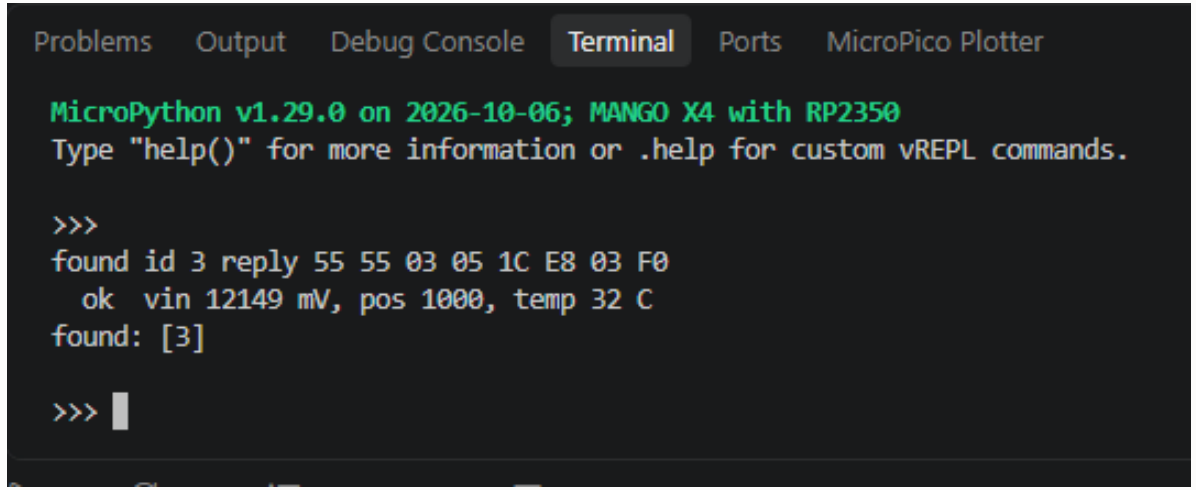



4

POWER_SW 撥到 ON

# 先讀，不要轉

在 VS Code 跑 scan.py · 掃描 ID 0–9，得到馬達 ID

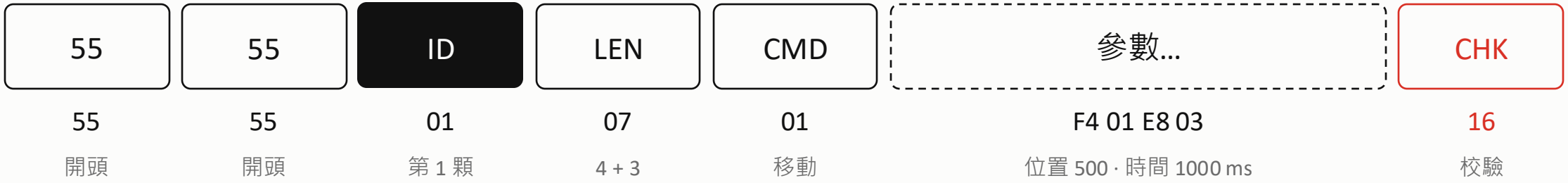



- ✓ 掃得到 ID
- ✓ 電壓約 12 V
- ✓ 溫度 30 °C 上下

掃不到？先檢查 XT30、POWER_SW、SS1 接頭有沒有插到底

記住這個 ID

# 一個封包長這樣




CHK = ~(ID + LEN + CMD + 參數) & 0xFF

固定 115200 baud · 數字低位元組在前（F4 01 = 0x01F4 = 500）· ID 254 是廣播

# 把封包送出去

servo.py

```
from machine import UART, Pin, PWM
import time
uart = UART(1, 115200, tx=Pin(8), rx=Pin(9))


def send(sid, cmd, params=b"", wait=30) -> bytes:
    body = bytes([sid, len(params) + 3, cmd]) + params

    while uart.any():
        uart.read(64)

    uart.write(b"\x55\x55" + body + bytes([~sum(body) & 0xFF]))

    time.sleep_ms(wait)
    n = uart.any()
    return (uart.read(min(n, 64)) or b"") if n else b""
```

UART1 就是 SS1 · 這塊板子收不到自己送出的回波

# 舵機能告訴你什麼？



## 電壓 · CMD 27

單位 mV，實測 12112



## 溫度 · CMD 26

單位 °C，實測 33



## 位置 · CMD 28

0–1000，實測 559



## 角度限制 · CMD 21

下限、上限，實測 0–1000

```
r = send(1, 27)   # read voltage of id 1
print(r[5] | r[6] << 8)  # 12112
```

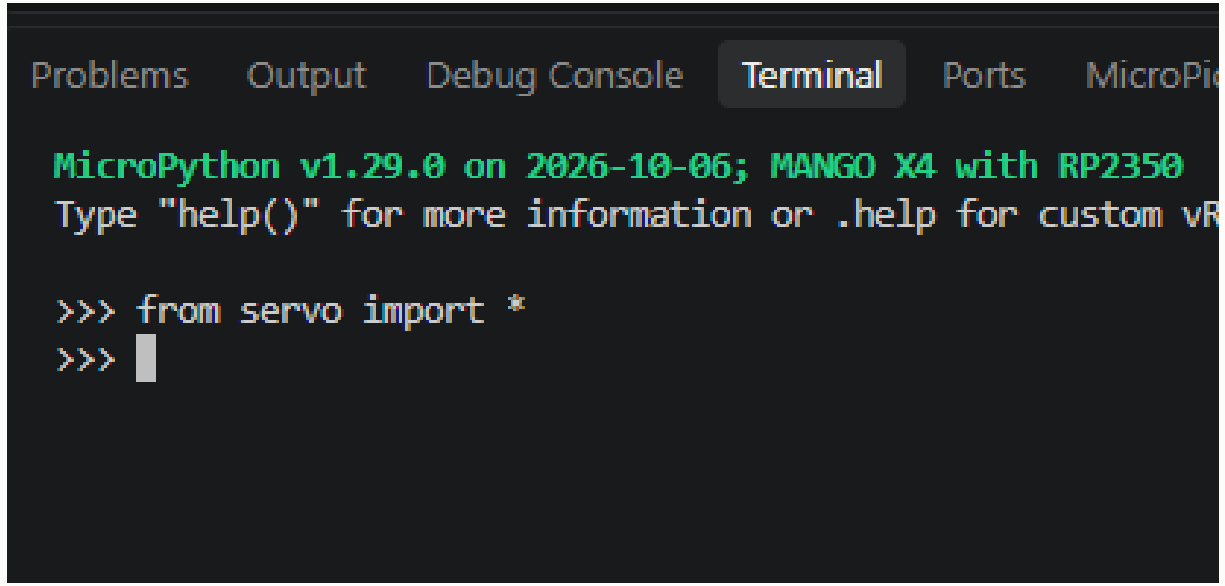
# 讓它轉

Upload servo.py 之後，可以使用這四個指令

| | |
|---|---|
| `move(id, 位置, 毫秒)` | 轉到指定位置，0–1000 |
| `pos(id)` | 讀現在的位置 |
| `vin(id)` | 讀電壓，單位 mV |
| `beep(n)` | 蜂鳴器響 n 聲 |

# 讓它轉






1

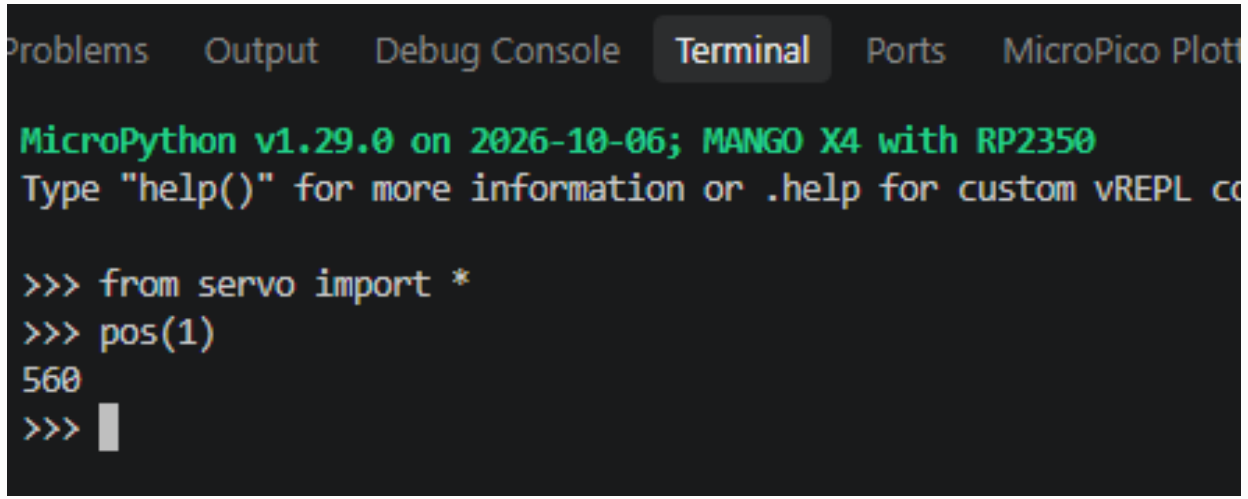
## 匯入指令

```
>>> from servo import *
```

每次重開板子都要打一次

# 讓它轉






2

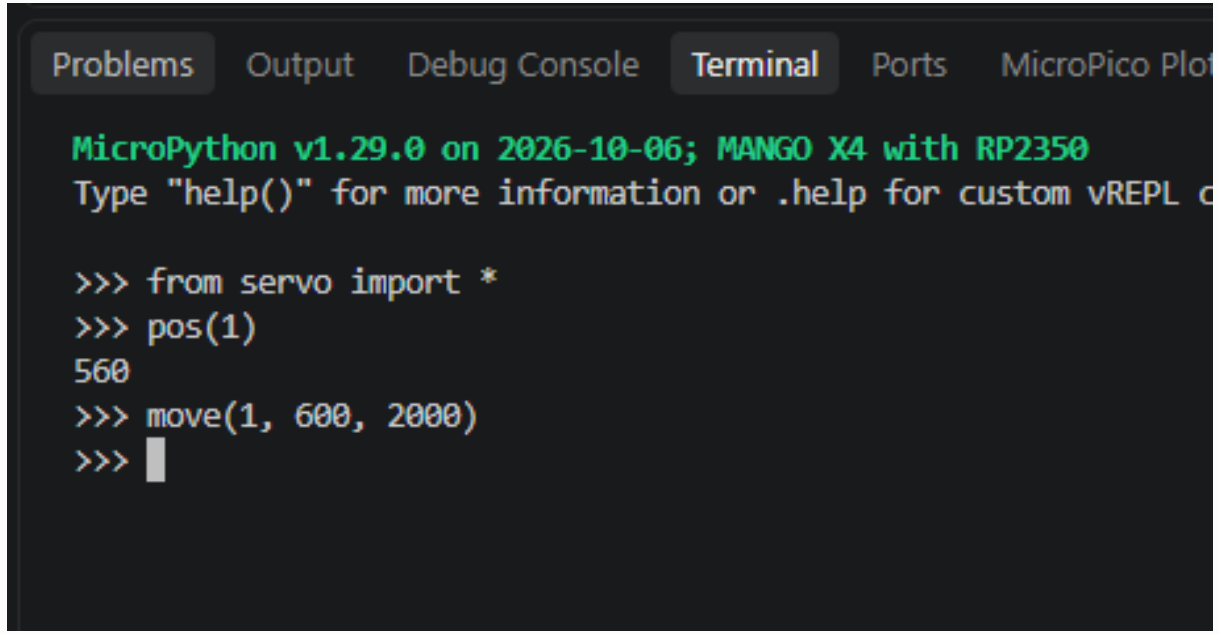
## 讀現在的位置

```
>>> pos(1)
560
```

先知道它停在哪裡

# 讓它轉






3

## 轉到 600

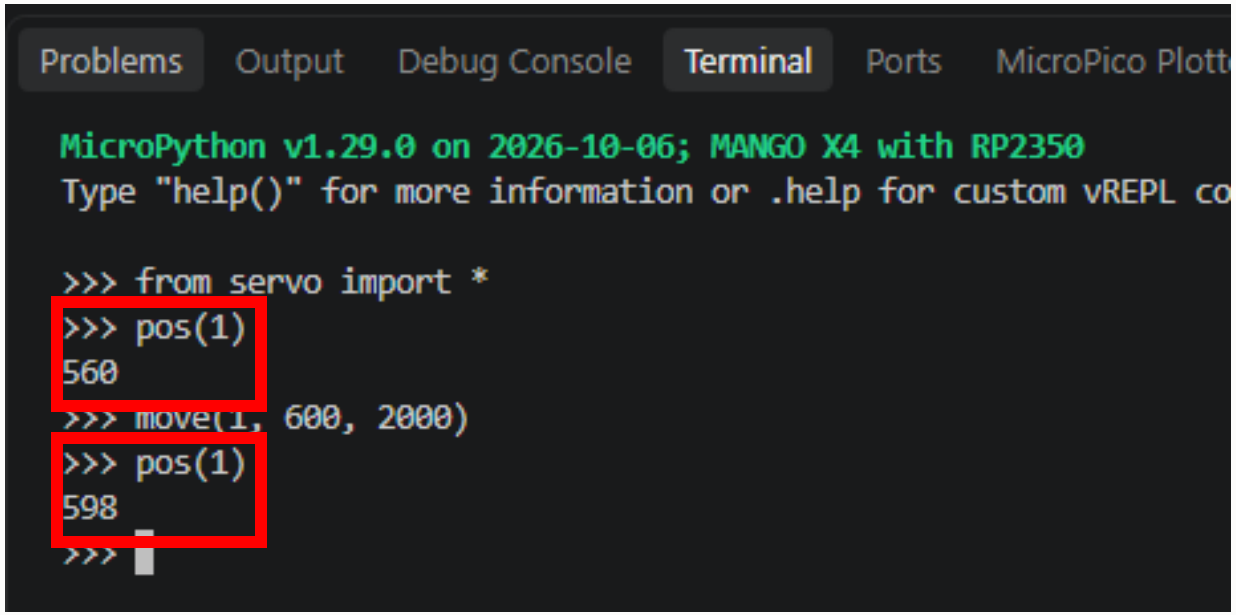
```
>>> move(1, 600, 2000)
```

2 秒轉到 600

手先固定好舵機

# 讓它轉






4

## 確認到了沒

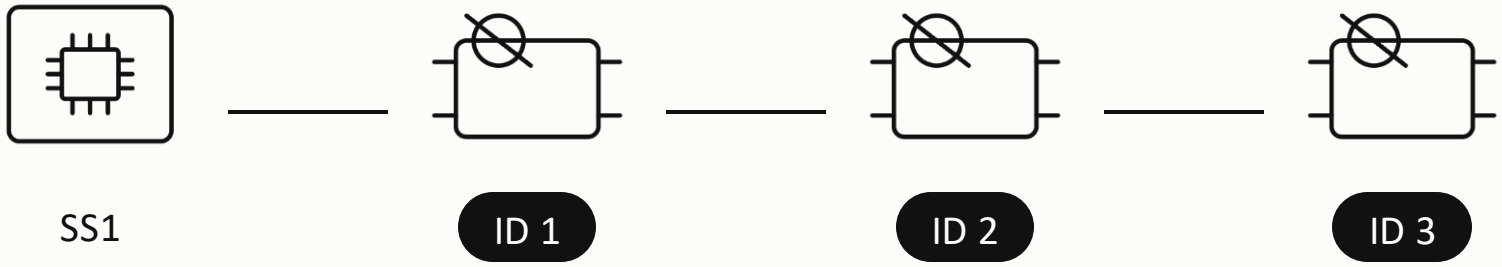
```
>>> pos(1)
598
```

差一點點是正常的

Part 2

# 多顆舵機

串接 · ID · 輪流轉


SS1
ID 1
ID 2
ID 3


Hiwonder BUS SERVO
45 HTD-45H
KG·cm

# ID 不同 vs ID 重複

## ID 都不同

只有指定的那顆回答

55 55 01 05 1C E8 03 F2

移動指令只轉那一顆

## 兩顆 ID 相同

兩顆同時回答，疊在一起

55 AA 05 20 02 28

移動指令兩顆一起轉

# 改 Servo ID





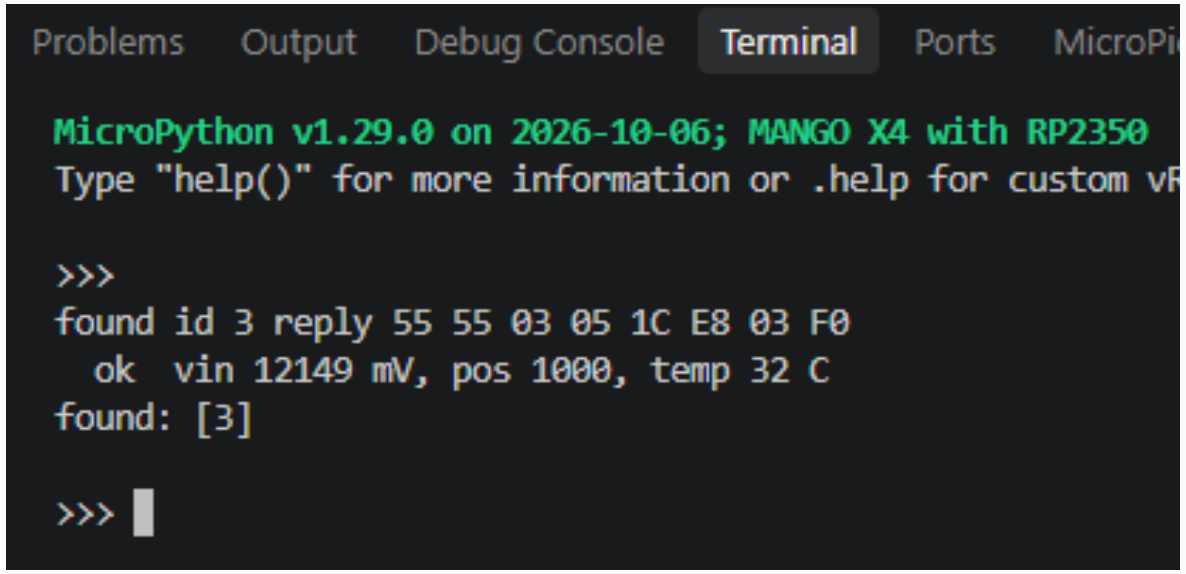

1

## 關電，只留一顆

重複的 ID 只能剩一顆在線上

# 改 Servo ID





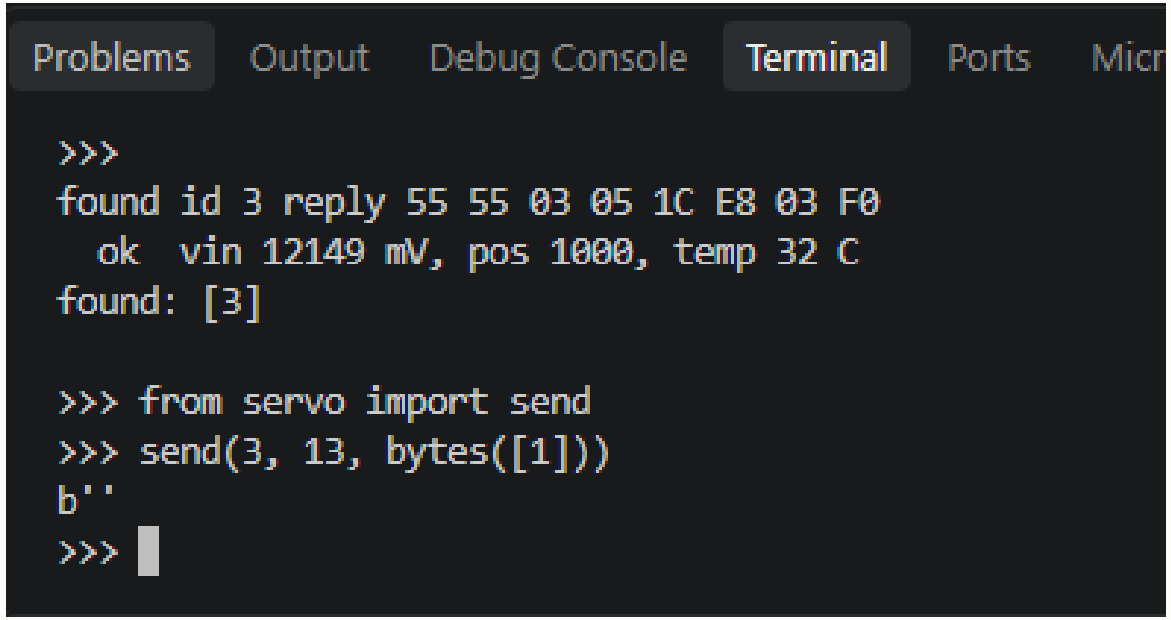

2

## 開電，讀讀看

執行 scan.py

# 改 Servo ID






3

## 送出改 ID

```
from servo import send
send(3, 13, bytes([1]))
```

例如把 ID 3 改成 1

# 改 Servo ID



```
>>> send(1, 27)
b'UU\x01\x05\x1bY/V'
>>>
```

4

## 用新 ID 讀

```
send(1, 27)
# 或是跑 scan.py
```

有回應就改好了

目標

# 將三個舵機 ID
# 改為 1, 2, 3

# 多顆一起轉

一步一步動起來（先 from servo import *）

```
move(2, 600, 1000)
```

只轉 ID 2

```
for i in (1, 2, 3): move(i, 600, 2000)
```

三顆一起轉到 600

```
for i in (1, 2, 3): move(i, 500, 2000)
```

三顆一起回中間

```
[pos(i) for i in (1, 2, 3)]
```

讀三顆的位置

# 轉不動？

讀電壓有回應嗎？ — 沒有 → 檢查 XT30、POWER_SW、SS1 接頭

有 ↓

某個 ID 的回覆是亂碼？ — 是 → ID 重複：只留一顆，改 ID

不是 ↓

沒送指令，GP9 也一直跳？ — 是 → **立刻斷電**，串接線一顆一顆加回去

不是 ↓

✓ 可以轉了

下一步

# 寫自己的舵機程式

# 寫自己的程式



example.py 的輸出

```
id 1 starts at 1000
    went to 951
    back at 999
id 2 starts at 0
    went to 49
    back at 1
id 3 starts at 560
    went to 511
    back at 559
```

1

## 打開 example.py，按 Run

這是個範例程式
響幾聲就是幾號在轉
每顆朝中間轉一點，再轉回來

# 寫自己的程式



play.py

```
from servo import *

# start your own code in here
beep()
```

2

## 換你寫：play.py

用 move / pos / vin / beep
寫出你自己的程式

# 寫自己的程式



挑戰：在 play.py 裡完成

- [ ] 讓一顆舵機來回擺 3 次
- [ ] 三顆同時轉
- [ ] 透過按鈕控制 Servo